agriculture

MDPI

Article

Research on Path Planning and Trajectory Tracking for Inspection Robots in Orchard Environments

Junlin Zhang [1,†], Longbo Su [1,†], Zhenhao Bai [1], Simon X. Yang [2], Ping Li [3], Shuangniu Hong [1], Weihong Ma [4] and Lepeng Song [1,*]

[1] School of Electronic and Electrical Engineering, Chongqing University of Science & Technology, Chongqing 401331, China; 2004030@cqust.edu.cn (J.Z.); 2023204006@cqust.edu.cn (L.S.); 2024204007@cqust.edu.cn (Z.B.); 2023204008@cqust.edu.cn (S.H.)
[2] Advanced Robotics and Intelligent Systems Laboratory, School of Engineering, University of Guelph, Guelph, ON N1G 2W1, Canada; syang@uoguelph.ca
[3] Chongqing Academy of Agricultural Sciences, Chongqing 400039, China; liping@cqaas.cn
[4] Information Technology Research Center, Beijing Academy of Agriculture and Forestry Sciences, Beijing 100097, China
* Correspondence: slphq@cqust.edu.cn
† These authors contributed equally to this work.

Abstract

In complex, semi-structured orchard environments, mobile inspection robots often suffer from excessive turning points, low search efficiency, limited trajectory-tracking accuracy, and poor adaptability to dynamic obstacles. To address these issues, this study proposes an integrated autonomous navigation method that employs an improved A* algorithm for global path planning, a Fuzzy-Weighted Dynamic Window Approach (FW-DWA) for local path optimization, and a model predictive control (MPC)-based trajectory-tracking controller. First, a dynamic heuristic-weight adjustment strategy is introduced into the conventional A* algorithm, in which a correction factor adaptively tunes the heuristic weight; a two-stage node optimization procedure then removes hazardous and redundant nodes to improve path smoothness and safety. Second, the FW-DWA, grounded in fuzzy control theory, uses goal distance and obstacle distance to update the weights of the heading, clearance, and velocity evaluation functions in real time, thereby enhancing obstacle avoidance in dynamic environments. Finally, a discrete kinematic model is established to design the MPC Controller, which achieves high-precision tracking through receding-horizon optimization and feedback correction. Experiments conducted in real orchards demonstrate that the proposed method reduces path length by 5.79%, shortens planning time by 3.64%, and increases the minimum safety distance by 50%. Comparative results further show that the MPC Controller attains a mean position error of 0.032 m and a mean heading error of 3.14°, clearly outperforming a conventional Proportional–Integral–Derivative (PID) controller. These findings provide an effective solution for reliable autonomous navigation of orchard inspection robots and offer a valuable reference for smart agricultural robotics applications.

Keywords: orchard inspection robot; path planning; fuzzy control; MPC; trajectory tracking

Check for updates

Academic Editor: José Lima

Received: 20 December 2025
Revised: 13 January 2026
Accepted: 9 February 2026
Published: 11 February 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

With the deep integration of the new generation of electronic information technology and agricultural systems engineering, traditional agricultural production is rapidly evolving toward modernization and intelligence. China is one of the world's major fruit-producing countries, but it is currently facing problems of labor shortage and rising labor

costs. Traditional manual inspection is inefficient and unreliable, and it can hardly meet the requirements of large-scale production. At the policy level, the development of intelligent inspection equipment is being promoted, and mobile robots have therefore become a natural choice [1,2]. However, orchards are semi-structured environments with complex terrain and highly variable obstacles, which poses significant challenges to autonomous navigation [3]. Luo et al. proposed a double-loop adaptive sliding mode control strategy to improve trajectory tracking in orchard robots. The approach reduces jitter, enhances disturbance rejection, and suppresses high-frequency chattering. Experimental results showed fast convergence of position and attitude errors, meeting the real-time and accuracy requirements for orchard mobile robots [4]. The robustness and accuracy of path-planning and trajectory-tracking technologies have become the core bottlenecks restricting the practical deployment of orchard inspection robots [5,6].

In the field of path planning, three main technical routes have been proposed: graph-search-based methods, sampling-based methods, and intelligent optimization algorithms [7]. Graph-search-based approaches such as Dijkstra, A*, and D* are widely used in static environments because they possess completeness and optimality [8]. Guo et al. introduced a weighted heuristic function and a key-point selection strategy into the A* algorithm, and used a second-order Bézier curve to smooth the planned path. They further combined the algorithm with DWA, where the global key points generated by the improved A* serve as interpolation points, thus achieving a fusion of global and local path planning [9]. Fu et al. improved the conventional A* algorithm by introducing a multi-neighborhood hybrid search strategy and dynamic weight coefficients, which effectively enhanced search efficiency and reduced redundant node expansion [10]. Zhang et al. proposed a robotic cherry harvesting solution using the Multi-Strategy Integrated RRT for Continuous Harvesting Path (MSI-RRTCHP) algorithm. The system uses machine vision to identify mature cherries and generate an optimal picking trajectory [11]. Huang et al. proposed an adaptive-neighborhood A* algorithm combined with a quadratic Bézier curve, which significantly improved path smoothness and stability [12]. Zhu et al. incorporated an obstacle influence factor and an adaptive weighting mechanism into the A* algorithm so that high-density obstacle regions could be actively avoided during the global search stage, and the generated path was smoothed using piecewise Bézier curves [13]. At the local level, the modified DWA fused a global guidance term with a distance-based heading-weight adjustment scheme, which effectively mitigated local minima and target-loss problems. Sampling-based methods such as RRT and its variants exhibit strong exploration ability in high-dimensional and complex environments. Dong et al. proposed FHQ-RRT, which improves the path-generation efficiency through dynamic sparse sampling and focused rewiring [14,15]. Cao et al. used Bayesian generalized kernel inference to construct a traversable map, converted the SOR-filtered 3D point cloud into a 2D occupancy grid, and then carried out path planning with the RRT* algorithm [16]. Sampling-based and optimization-based planners excel in open environments but struggle in semi-structured, resource-limited orchards. The former's path stochasticity risks instability in narrow rows, while the latter's computational intensity challenges the real-time capabilities of cost-effective embedded systems. In terms of intelligent optimization, Sun et al. introduced six improvements to the traditional ant colony optimization (ACO), including nonuniform pheromone initialization, a direction-aware heuristic, an adaptive pheromone evaporation factor, a differentiated pheromone update rule, an improved state-transition probability strategy, and a path-smoothing mechanism, so as to solve the problems of slow convergence and premature stagnation in classical ACO [17]. Wu et al. proposed a hybrid path-planning algorithm that combines an adaptive-step-size RRT with an improved ACO [18]. By generating an initial path to optimize the pheromone distribution and reduce the search space,

the algorithm significantly accelerated convergence. At the same time, a reinforced state-transition rule, path-crossing strategy, improved pheromone update mechanism, and local optimization strategy were introduced to further enhance planning performance. Li et al. improved the solution accuracy and convergence speed of PSO by introducing a nonlinear decreasing inertia weight and adaptive learning factors, and they considered both path length and smoothness in the fitness function to address the problems of overly long and nonsmooth paths in traditional algorithms [19]. Li et al. further combined TOACA with DWA to form the TOACA-DWA method, which, through the introduction of a dynamic penalty factor and key-node filtering mechanism, realized efficient obstacle avoidance in dynamic environments [20]. Although the above methods have all made progress, traditional planners still suffer from model mismatch, susceptibility to local optima, and insufficient real-time performance when deployed in semi-structured, dynamically changing, and spatially constrained orchard scenes. In particular, under narrow orchard passages and irregular obstacle distributions, algorithms that rely solely on grid maps or idealized scenes are often unable to balance global optimality and local feasibility [21].

In the field of trajectory-tracking control, researchers have proposed a variety of control strategies for the nonlinear dynamics of mobile robots and for environmental disturbances [22]. Traditional PID controllers are widely applied because of their simple structure and ease of implementation, but their performance becomes limited in complex dynamic environments and under time-varying parameters. Li et al. proposed a control architecture that combines a backstepping controller (BSC) with a fractional-order PID controller and used an improved grey wolf optimization algorithm for parameter tuning, which significantly improved system stability and tracking accuracy [23]. Huang et al. designed a variable-structure controller that combines sliding-mode control (SMC) with backstepping [24]. By introducing different sliding surfaces, the method achieved rapid convergence of tracking errors and exhibited higher robustness than conventional methods. However, the inherent chattering of SMC and its implementation complexity limit its practical applications. Yiğit et al. used the A* algorithm to obtain an optimal path so as to improve the efficiency and flexibility of the planning stage [25]. In terms of controller design, they first introduced kinematics-based backstepping control (KBBC) to enhance control efficiency, then combined SMC and PID control strategies to reduce the tracking errors in position coordinates and heading angle between the desired trajectory and the actual motion. Yagmur et al. combined pole placement, PID, and LQR control strategies to design five optimized control models and, for the first time, introduced the clonal selection algorithm (CSA) to adaptively optimize multiple PID parameters, thereby providing stronger global search capability and robustness [26]. Fan et al. exploited the dynamic interaction ability of dual Q-learning to realize online adaptive tuning of MPC parameters and introduced an incremental nonuniform discretization strategy to adaptively partition the action and state spaces, thus improving learning efficiency [27]. Liu et al. proposed an adaptive model predictive control-based command planner and a multi-terrain trajectory-tracking framework. This method aims to address the difficulty of traditional tracking models in adapting to multiple terrains and uncertain dynamics, such as those encountered by spherical robots [28]. Tang et al. proposed a geometric MPC method that takes the manifold characteristics and motion constraints of the robot into account in controller design and solves the control problem using Lie-group and Lie-algebra formulations [29]. Zhang et al. addressed the problems of insufficient steady-state accuracy and degraded dynamic performance in repetitive trajectory tracking of autonomous agricultural vehicles under unknown iterative disturbances by introducing three modules, namely an adaptive high-order internal model, the internal model principle, and an input-constrained auxiliary-variable design, thereby achieving overall system performance enhancement [30].

Xu et al. proposed a dynamic MPC scheme that can adaptively adjust the prediction horizon to achieve higher tracking accuracy and lower computational complexity in complex paths [31]. Ma et al. developed a specialized robot for inspecting laying hens in large-scale egg farms. The robot uses deep learning for real-time detection, QR code-based positioning for localization, and autonomous navigation for efficient farm movement [32].

In summary, orchard inspection robots face significant challenges regarding path planning and trajectory tracking in complex orchard environments, particularly characterized by narrow passages, irregular obstacles, and dynamic changing conditions. Traditional methods exhibit limitations in addressing these features; path planning and trajectory tracking are often treated as decoupled tasks, lacking effective global–local coupling. This frequently results in redundant turning points, compromised obstacle avoidance capabilities, and an inability to balance safety with efficiency. Moreover, existing control methods often demonstrate suboptimal performance when handling model uncertainties and external disturbances, coupled with a lack of path node optimization and smoothing, which constrains the overall system effectiveness. To address these issues, this study proposes an integrated path-planning method aimed at solving complex navigation problems in orchard environments. By establishing a collaborative optimization framework among dynamic heuristic function adjustment, fuzzy weighting, and MPC, this method enhances the overall efficiency of path planning, obstacle avoidance, and trajectory tracking. Specifically, the dynamic heuristic function adaptively optimizes path search weights based on real-time environmental feedback, improving algorithmic performance under varying conditions. Fuzzy control theory is integrated into the DWA to dynamically regulate evaluation weights according to obstacle density and distribution, effectively strengthening obstacle avoidance capabilities. Furthermore, MPC-based trajectory tracking ensures precise tracking along the optimized path through receding horizon optimization and real-time feedback correction, thereby improving accuracy and stability. The synergy among these three components enables the proposed method to achieve efficient path planning and precise navigation in complex orchard environments, providing a reliable solution for autonomous navigation.

2. Materials and Methods

2.1. Overall Scheme Design

Driven by the research needs of path planning and trajectory tracking for orchard inspection robots, this work establishes a complete technical framework spanning from hardware platform setup to intelligent decision-making and execution. As shown in Figure 1, the overall scheme centers on four core stages: mobile platform integration, mapping and data transmission, path-planning algorithm design, and trajectory tracking and execution.

The mobile platform employs a heterogeneous dual-core architecture. The RDK X5 serves as the high-level controller for computationally intensive tasks—including LiDAR point cloud processing, improved A* planning, and MPC solving—utilizing multithreading to ensure a perception–decision cycle within 500 ms. The STM32F407 acts as the low-level execution unit for deterministic tasks such as sensor data acquisition, IMU fusion, and motor PID control. To enhance robustness in harsh orchard environments, a hierarchical fault-tolerant mechanism is implemented. This includes real-time sensor monitoring that triggers deceleration or standby modes upon data anomalies, and a communication protection scheme via high-speed UART; if the STM32 detects a command timeout through the watchdog mechanism, it automatically executes emergency braking to prevent loss of control. For mapping and sensing, a multi-sensor fusion scheme is adopted. LiDAR and vision sensors acquire point-cloud data and obstacle information of the orchard environment. On this basis, a grid-mapping algorithm converts the raw sensor data into a

2D occupancy grid map suitable for path planning, and map visualization with real-time updates is implemented via Microsoft Foundation Class (MFC). At the path-planning layer, the system employs an improved trajectory-planning algorithm. Through cost-map construction, removal of unconditional coefficients, global risk-factor evaluation, and reverse adjustment, it generates an optimal path that balances safety and efficiency. The algorithm jointly considers tree density, terrain undulation, and dynamic obstacles to ensure the planned path both avoids obstacles and shortens travel distance. In the trajectory-tracking execution stage, a real-time tracking controller is designed within a model predictive control framework. By optimizing the objective function and specifying constraints, the controller enables precise following of the planned path. It dynamically adjusts the robot's linear and angular velocities to maintain stable tracking performance in narrow orchard aisles and complex road conditions. Finally, field experiments in orchards verify the feasibility and superiority of the overall scheme for practical operations, providing reliable technical support for intelligent orchard management. The mathematical formulations of the kinematic model, the improved A* search, the FW-DWA, and the MPC-based tracker are detailed in Sections 2.2–2.5 to maintain the clarity of the system-level description.

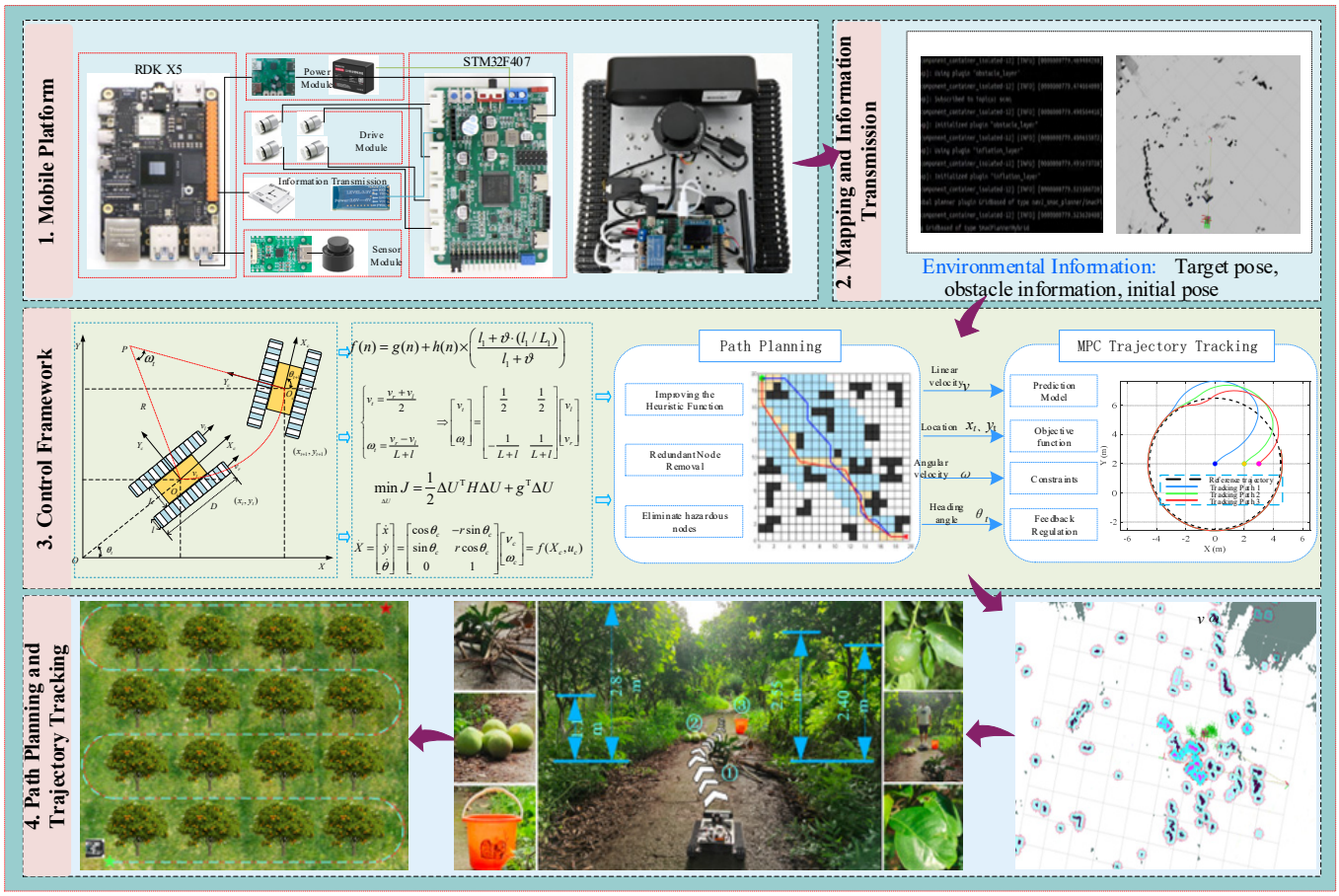


Figure 1. Overall system design.

This study proposes a hierarchical collaborative architecture comprising a 'Global Planner–Local Planner–MPC Controller' to achieve efficient path planning and precise trajectory tracking in dynamic orchard environments. The global planning layer generates an optimal collision-free reference path based on the static grid map. The local planning layer utilizes real-time sensory data to optimize the trajectory online, addressing dynamic obstacles and local environmental changes. To harmonize the interactions between these layers, a local–global collaborative re-planning mechanism is introduced: global re-planning

is triggered only when the local path deviation exceeds a preset threshold; otherwise, high-frequency local updates are executed to balance response speed and computational efficiency. Subsequently, the MPC layer takes the Local Planner's output as the reference trajectory. By integrating the robot's current state with kinematic constraints, the MPC optimizes control inputs via receding horizon control to achieve high-precision tracking. When trajectory deviation increases, the MPC actively suppresses error accumulation, ensuring the robot smoothly converges back to the global reference path. The detailed workflow of this architecture is illustrated in Figure 2.

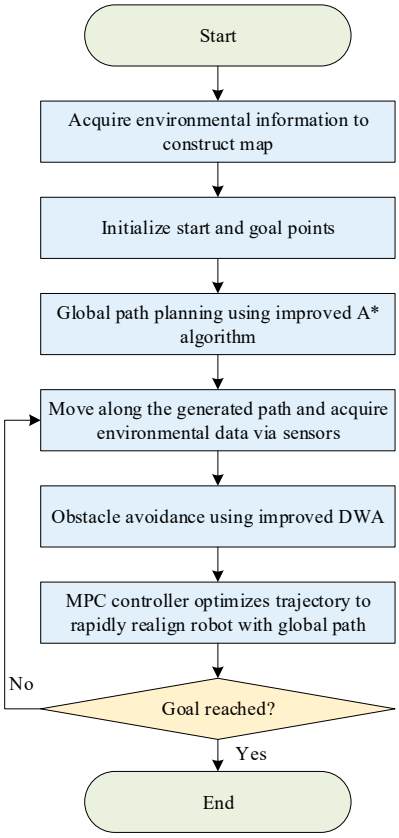


Figure 2. Architecture and workflow of the hierarchical collaborative navigation system integrating Global Planner, Local Planner, and MPC Controller.

2.2. Kinematic Model of the Tracked Mobile Robot

According to the kinematic equations of the orchard mobile robot, given that the orchard ground is predominantly soil and its undulation is within 0.15 m, the ground is treated as a flat two-dimensional plane for simplified modeling. The tracked robot adopts a rear-driven differential-steering configuration and mainly consists of the chassis and the tracked propulsion units. To construct its mathematical model, a global coordinate frame XOY is first defined to measure its spatial position. Meanwhile, a local frame $X_C O_C Y_C$ is established and fixed at the robot's center of mass to describe its pose. For simplicity, the following assumptions are made: the tracks do not deform in length; the pressure distribution over the track–ground contact patch is uniform; and the center of mass is located at the center of the chassis. In addition, the tracked mobile robot achieves differential control by regulating the left and right track velocities, which correspondingly simplifies its kinematic model.

In Figure 3, point C denotes the robot's center with coordinates (x_t, y_t), and the robot's heading angle is θ_t. The axes X_c and Y_c represent the directions aligned with the robot's forward motion and its perpendicular (lateral) direction, respectively.

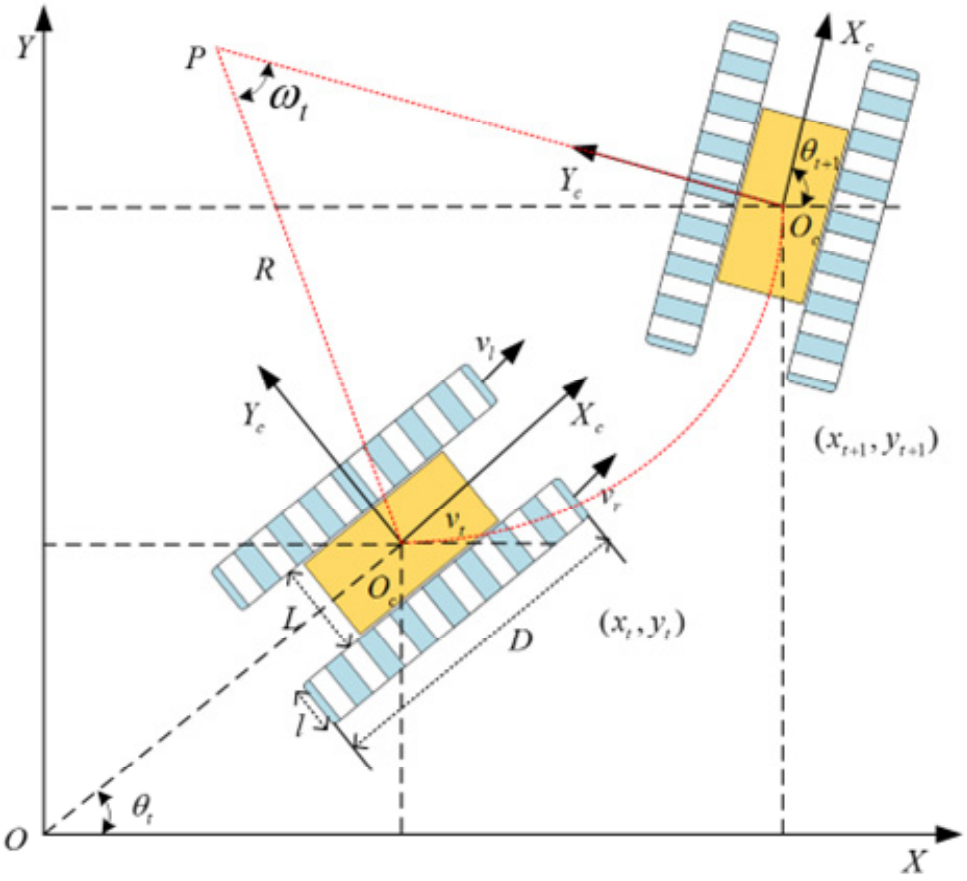


Figure 3. Robot kinematic model. Note: XOY denotes the global planar coordinate frame; $X_cO_cY_c$ denotes the body-fixed (vehicle) frame of the orchard mobile robot; O_c is the geometric center of the robot; x_t and y_t are the coordinates of the robot's center of mass along the X- and Y- axes, respectively; L is the chassis width; l is the width of a single track on each side; D is the length of a single track on each side; P is the rotational reference point; ω_t is the angular (rotational) velocity; v_t is the chassis forward velocity; v_l is the left-track forward velocity; v_r is the right-track forward velocity; θ_t is the robot's heading angle; R is the radius of the motion trajectory.

The robot state at any time can thus be written as $P(x_t, y_t, \theta_t)$. Therefore, the robot's kinematic equations can be expressed as

$$\begin{cases} \dot{x}_t = v_t \cos\theta_t \\ \dot{y}_t = v_t \sin\theta_t \\ \dot{\theta}_t = \omega_t \end{cases} \tag{1}$$

For the tracked mobile robot platform studied in this paper, the control input u must be converted into the left and right track velocities to effectively control the robot's motion. Under differential-drive operation, the robot's linear velocity v_t and angular velocity ω_t are related to the left and right track speeds as follows:

$$\begin{cases} v_t = \dfrac{v_r + v_l}{2} \\ \omega_t = \dfrac{v_r - v_l}{L + l} \end{cases} \Rightarrow \begin{bmatrix} v_t \\ \omega_t \end{bmatrix} = \begin{bmatrix} \dfrac{1}{2} & \dfrac{1}{2} \\ -\dfrac{1}{L+l} & \dfrac{1}{L+l} \end{bmatrix} \begin{bmatrix} v_l \\ v_r \end{bmatrix} \tag{2}$$

Because the slip parameters of a tracked robot are influenced by payload, ground contact conditions, and motion speed, their identification is challenging. Accordingly, this

study simplifies the tracks to a two-wheel differential-drive model to facilitate further research on path planning and trajectory tracking; the specific kinematic model is given in Equation (1).

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta}_t \end{bmatrix} = \begin{bmatrix} \cos\theta_t & -r\sin\theta_t \\ \sin\theta_t & r\cos\theta_t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ -\frac{1}{L+l} & \frac{1}{B} \end{bmatrix} \begin{bmatrix} v_l \\ v_r \end{bmatrix} = \begin{bmatrix} \frac{\cos\theta_t}{2} - \frac{r\sin\theta_t}{L+l} & \frac{\cos\theta_t}{2} + \frac{r\sin\theta_t}{L+l} \\ \frac{\sin\theta_t}{2} + \frac{r\cos\theta_t}{L+l} & \frac{\sin\theta_t}{2} + \frac{r\cos\theta_t}{L+l} \\ -\frac{1}{L+l} & \frac{1}{L+l} \end{bmatrix} \dot{u} \tag{3}$$

Assuming that the inspection robot has a turning radius R during steering, then

$$R = \frac{v_t}{\omega_t} = \frac{L+l}{2} \cdot \frac{v_r + v_l}{|v_r - v_l|} \tag{4}$$

We define velocities in the vehicle's forward direction as positive. When $v_l > v_r$, the left track moves faster than the right and the vehicle turns clockwise; when $v_r > v_l$, the right track moves faster than the left and the vehicle turns counterclockwise. When $v_l = v_r$, both tracks have the same speed and the vehicle travels in a straight line; when $v_l = -v_r$, the two tracks have equal speeds in opposite directions, so the turning radius is $R = 0$, and the vehicle rotates in place. A schematic of the inspection robot's steering is shown in Figure 4.

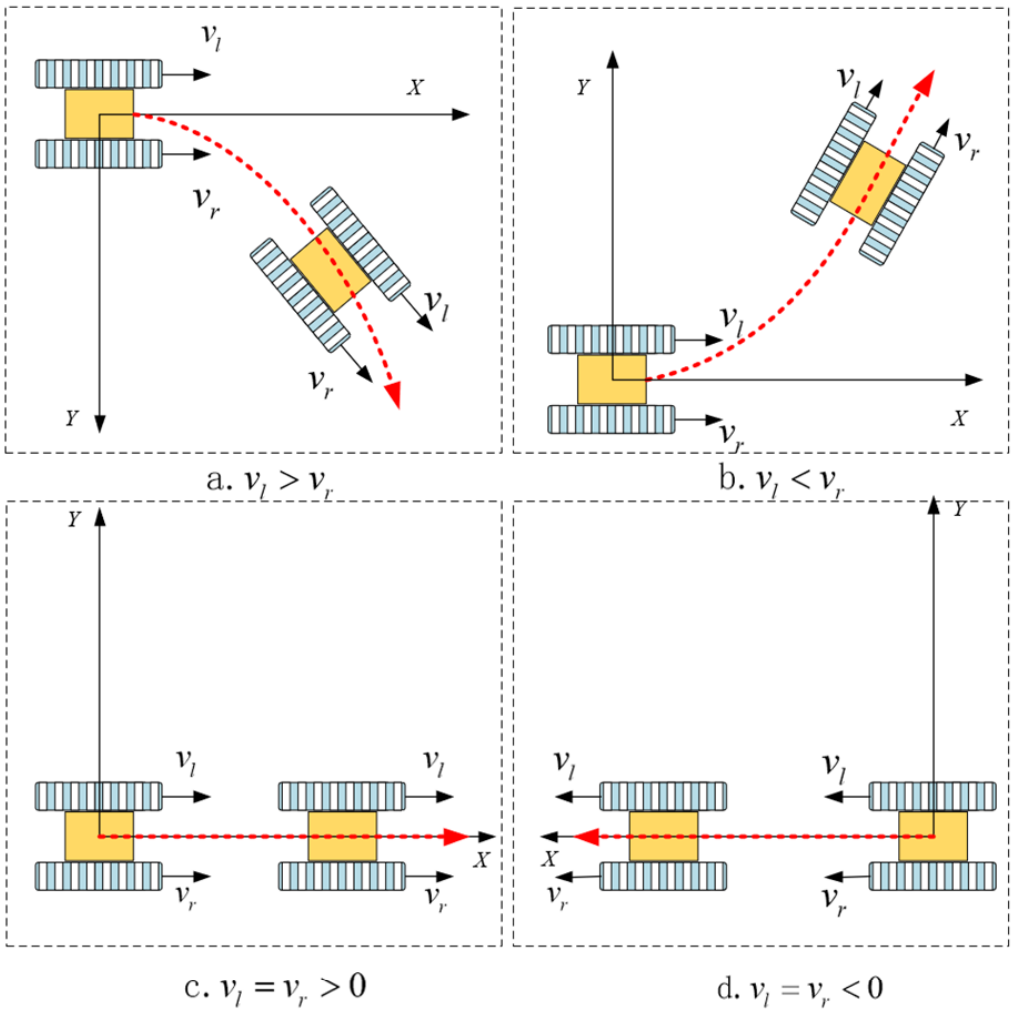


Figure 4. Schematic diagram of orchard inspection robot steering.

2.3. An Improved A Algorithm for Global Path Planning*

2.3.1. Optimization Strategy for Heuristic Function

The A* algorithm is a search method widely used for path planning; its core idea is to evaluate each node's priority using the cost function f(n) so as to obtain the optimal path. The basic formulation of A* is

$$f(n) = g(n) + h(n) \tag{5}$$

where $g(n)$ denotes the actual cost from the start node to the current node n, and $h(n)$ is the heuristic function that estimates the minimum distance from n to the goal node. This combination enables A* to effectively evaluate the total path cost and use heuristic information to guide the search process, thereby finding the optimal path in complex spaces. The heuristic function $h(n)$ is central to the behavior of A*. Overestimation of $h(n)$ (i.e., loss of admissibility) can reduce node expansions and improve runtime, but the returned path may no longer be optimal. Conversely, underestimation weakens guidance, enlarges the search frontier, and increases computational cost. Ideally, $h(n)$ should equal or closely approximate the true cost-to-go from the current node n to the goal. To enhance A*, we introduce a heuristic-weight adaptation scheme that dynamically regulates the contribution of $h(n)$ during search. The new formulation is

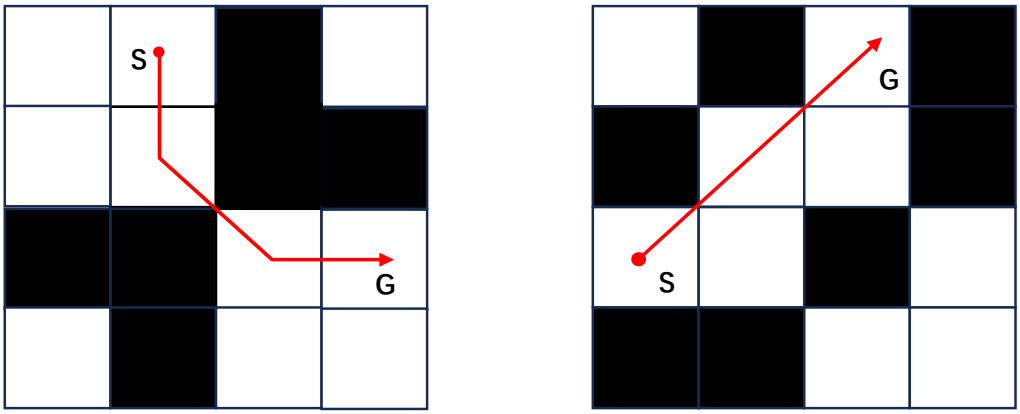
$$f(n) = g(n) + h(n) \times \left(\frac{l_1 + \vartheta \cdot (l_1 / L_1)}{l_1 + \vartheta} \right) \tag{6}$$

In this new formulation, l_1 denotes the straight-line (Euclidean) distance from the current node to the goal, whereas L_1 represents the overall distance from the start to the goal. By introducing a tuning factor ϑ, the formulation provides a flexible dynamic adjustment mechanism that can automatically optimize the heuristic function according to the current search state. When the current node is far from the goal, the ratio l_1 / L_1 is large, which in practice increases the weight of $h(n)$. This biases the search toward expanding nodes closer to the goal, thereby accelerating the discovery of a feasible path. Conversely, when the current node is near the goal, the ratio l_1 / L_1 decreases and the weight on $h(n)$ is reduced; the algorithm then places greater emphasis on the accumulated cost $g(n)$ from the start to the current node.

2.3.2. Optimization Method for Path Nodes

Paths planned by the conventional A* algorithm often contain too many turning points, exhibit poor curvature smoothness, and may pass through hazardous child nodes. To mitigate these issues, representative nodes are selected from the set of confirmed nodes to serve as the final path nodes; a rule-based filtering procedure then removes unnecessary nodes, thereby reducing the number of turns. The resulting path remains collision-free with respect to obstacles while eliminating redundant waypoints to shorten the overall path. The optimization objective is to eliminate path risks, particularly to prevent the path from obliquely intersecting obstacle vertices. This issue is clearly illustrated in Figure 5a,b; in this context, S denotes the starting point, and G denotes the endpoint.

Figure 5a shows a path that passes directly through the vertex of an obstacle, which could damage the robot during inspection operations. A more severe case, shown in Figure 5b, has the path obliquely traversing the gap between the vertices of two obstacles, which is unacceptable in practice. Therefore, path planning must account for the possibility of such cases and incorporate appropriate strategies to avoid these risks.



(**a**) Hazardous Path 1 (**b**) Hazardous Path 2

Figure 5. Example of hazardous paths.

As shown in Figure 6, to address the above problem, targeted measures can be taken according to the obstacle's relative position. When the obstacle is at relative position A, the upper and lower child nodes should be removed. When it is at position B, there is no risk of crossing an obstacle vertex, so no action is required. When the obstacle is at position C, the left and right candidate child nodes should be deleted to avoid traversing along the obstacle's horizontal edge.

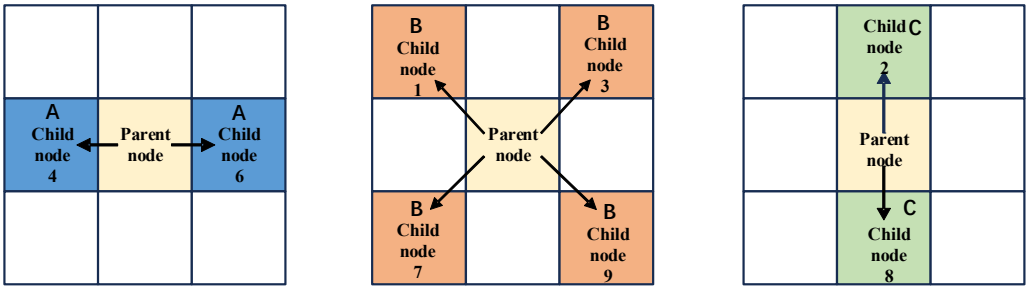
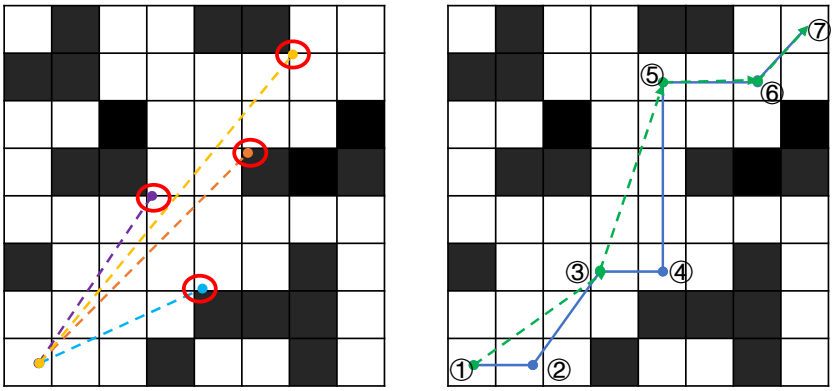


Figure 6. Schematic diagram of various obstacles.

In addition to eliminating hazardous paths, the removal of redundant nodes constitutes a critical component of path optimization. By eliminating risk nodes and redundant inflection points, the algorithm optimizes the geometric continuity and safety of the path while enhancing trackability. This effectively minimizes excessive corrections during the local planning phase, thereby providing a more consistent and executable reference path for subsequent trajectory tracking. Taking the Figure 7 above as an example, where ① denotes the starting point and ⑦ the goal, the traditional A* algorithm would evaluate the costs of neighboring nodes to incrementally select a path. However, inspection of the figure reveals that the traditional path ①→②→③→④→⑤→⑥→⑦ is suboptimal; the ideal trajectory should be ①→③→⑤→⑥→⑦. By adopting specific optimization strategies, the path can proceed directly from ① to ③ and subsequently to ⑤, bypassing the unnecessary nodes ② and ④. This modification significantly reduces the total path length and improves algorithmic efficiency.



(**a**) Hazardous Path Removal (**b**) Redundant Node Pruning Strategy

Figure 7. Hybrid path optimization in grid-based orchard environments.

2.4. Fuzzy Weighted DWA for Local Path Planning

2.4.1. Traditional DWA Framework

The DWA formulates local obstacle avoidance and path following as an online optimization in velocity space. Within a dynamic window jointly defined by velocity and acceleration constraints, it discretely samples candidate velocities; using the robot's kinematic/dynamic model, it performs short-horizon forward rollouts for each candidate to generate a set of trajectories; an aggregate evaluation function scores the trajectories and selects the best one, and the first control input of the optimal trajectory is applied in the next control cycle. In this way, DWA rapidly computes feasible, high-quality actions in velocity space with low computational overhead, ensuring flexible and safe motion while satisfying the imposed constraints.

When applying the DWA, the motion trajectory prediction relies on the robot's kinematic model. As the differential kinematic model has been explicitly established in Section 2.2, we directly utilize it to simulate the robot's trajectory over the prediction horizon, avoiding repetitive mathematical derivation. The DWA reformulates obstacle avoidance for inspection robots as a constrained optimization in velocity space. It primarily accounts for three classes of constraints: the robot's own velocity limits, its dynamic-performance limits, and a safety braking-distance constraint. For each velocity sample (v, w), an objective function evaluates its quality. This function typically consists of three terms: a *heading* score for progress toward the goal, a *dist* score for obstacle clearance, and a *velocity* score for speed maintenance.

$$J(v, w) = \alpha \cdot heading(v, w) + \beta \cdot dist(v, w) + \gamma \cdot velocity(v, w) \tag{7}$$

where α, β, γ are weighting factors. α is the weight for the heading evaluation term and is closely related to the robot's direction of travel; β is the weight for the safety-distance term, affecting the distance maintained between the mobile robot and surrounding obstacles; and γ is the weight associated with the robot's speed.

2.4.2. Fuzzy Controller Design

Through in-depth study and experimentation on the conventional DWA, we found that the settings of the evaluation-function weights α, β, γ are critical to obstacle-avoidance performance. Accordingly, this section proposes a strategy that adaptively links these three weights to the environment to improve static obstacle avoidance. In addition, we integrate the traditional DWA with fuzzy control theory to form a FW-DWA. Within this fuzzy control system, DWA can dynamically adjust the weight coefficients in the evaluation function, thereby selecting a more favorable avoidance path.

The designed fuzzy controller adopts a two-input, three-output structure. The inputs are the robot-to-goal distance (D) (universe of discourse ([0, 6])) and the robot-to-obstacle distance O (universe ([0, 2])), each partitioned into three fuzzy sets: Near (N), Medium (M), and Far (F). The outputs are the control coefficients α, β, γ (universe ([0, 1])), partitioned into five fuzzy sets: Extra-Small (XS), Small (S), Medium (M), Large (L), and Extra-Large (XL). The form is determined via empirical tuning to balance control performance against computational load, ensuring the optimization problem remains solvable within the 0.1 s cycle on the embedded hardware. Membership functions are selected due to their low computational complexity, which satisfies the requirements for rapid real-time inference on embedded systems in dynamic environments. The fuzzy rule base constructed in this study is shown in Table 1.

Table 1. Fuzzy rule table.

Serial No.	Input Variables		Output Variables		
	Distance to Target	Distance to Obstacles	α	β	γ
1	N	N	S	XL	XS
2	N	M	S	L	S
3	N	F	S	M	M
4	M	N	M	XL	XS
5	M	M	M	M	M
6	M	F	M	S	L
7	F	N	L	XL	XS
8	F	M	L	L	M
9	F	F	XL	S	XL

Based on the sensor information, the fuzzy rule base is applied and fuzzy inference is performed using the membership functions. Defuzzification is carried out with the mean-of-maxima (MOM) method, converting the fuzzy outputs into specific numerical values for the control coefficients α, β, γ.

2.5. MPC-Based Trajectory-Tracking Controller Design

2.5.1. Prediction Model

Based on the kinematic model presented in Section 2.2, we construct a pose-error model and, on this basis, implement a predictive trajectory-tracking controller. Let the robot's desired pose be $P(x_c, y_c, \theta_c)$, where (x_c, y_c) denotes the desired coordinates and θ_c the desired heading angle. Accordingly, the desired trajectory can be expressed according to Equation (8).

$$\dot{X} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos\theta_c & -R\sin\theta_c \\ \sin\theta_c & R\cos\theta_c \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \nu_c \\ \omega_c \end{bmatrix} = f(X_c, u_c) \tag{8}$$

During trajectory-tracking control, the errors between the actual and desired control inputs and between the actual and desired poses are assumed to be small. Here, X represents the current state, X_c represents the desired state, u represents the actual control input, and u_c represents the desired control input. With the robot's state vector and the control input linearizing the dynamics, we perform a first-order Taylor expansion of the right-hand side $f(X_c, u_c)$ about the desired trajectory and compute the Jacobians $\partial f(X_c, u_c)/\partial X_c$ and $\partial f(X_c, u_c)/\partial u_c$. We define the state deviation $\widetilde{X} = X - X_c$ and the input deviation $u - u_c = \widetilde{u}$; then the increment of the state-error dynamics is given by

$$\begin{bmatrix} \dot{x} - \dot{x}_c \\ \dot{y} - \dot{y}_c \\ \dot{\theta} - \dot{\theta}_c \end{bmatrix} = \begin{bmatrix} 0 & 0 & -v_c\sin\theta_c \\ 0 & 0 & v_c\cos\theta_c \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x - x_c \\ y - y_c \\ \theta - \theta_c \end{bmatrix} + \begin{bmatrix} \cos\theta_c & 0 \\ \sin\theta_c & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v - v_c \\ \omega - \omega_c \end{bmatrix} \tag{9}$$

$$\dot{\widetilde{X}} = A\widetilde{X} + B\widetilde{u} \tag{10}$$

where $A = \begin{bmatrix} 0 & 0 & -v_c \sin\theta_c \\ 0 & 0 & v_c \cos\theta_c \\ 0 & 0 & 0 \end{bmatrix}$, $\mathrm{B} = \begin{bmatrix} \cos\theta_c & 0 \\ \sin\theta_c & 0 \\ 0 & 1 \end{bmatrix}$,

v_c is the desired linear velocity, w_c is the desired angular velocity, and θ_c denotes the desired heading angle.

The above equations constitute the continuous linear state-space model of the orchard inspection robot. T is the sampling time, applying the Euler discretization and rearranging terms yields the corresponding discrete state-space equations:

$$\widetilde{X}(k+1) = (TA + E)\widetilde{X}(k) + TB\widetilde{u}(k) = \begin{bmatrix} 1 & 0 & -Tv_c \sin\theta_c \\ 0 & 1 & Tv_c \cos\theta_c \\ 0 & 0 & 1 \end{bmatrix} \widetilde{X}(k) + \begin{bmatrix} T\cos\theta_c & 0 \\ T\sin\theta_c & 0 \\ 0 & T \end{bmatrix} \widetilde{u}(k) \tag{11}$$

Let

$$\alpha_1 = \begin{bmatrix} 1 & 0 & -Tv_c \sin\theta_c \\ 0 & 1 & Tv_c \cos\theta_c \\ 0 & 0 & 1 \end{bmatrix}, \beta_1 = \begin{bmatrix} T\cos\theta_c & 0 \\ T\sin\theta_c & 0 \\ 0 & T \end{bmatrix}, \xi(k) = \begin{bmatrix} \widetilde{X}(k) \\ \widetilde{u}(k-1) \end{bmatrix}.$$

Among these, α_1 is the discrete state matrix, β_1 is the discrete input matrix, and $\xi(k)$ is the augmented state vector.

Therefore, the new state-space formulation can be expressed as

$$\xi(k+1) = \begin{bmatrix} \alpha_1 \widetilde{X}(k) + \beta_1 \widetilde{u}(k-1) \\ \widetilde{u}(k-1) \end{bmatrix} + \begin{bmatrix} \beta_1 \widetilde{u}(k) - \beta_1 \widetilde{u}(k-1) \\ \widetilde{u}(k) - \widetilde{u}(k-1) \end{bmatrix} \tag{12}$$

Let $\Delta u = \widetilde{u}(k) - \widetilde{u}(k-1)$; then $\xi(k+1)$ can be expressed as

$$\xi(k+1) = D_1 \xi(k) + E\Delta\widetilde{u}(k) \tag{13}$$

where $D_1 = \begin{bmatrix} \alpha_1 & \beta_1 \\ 0 & I_{Nu} \end{bmatrix}, E = \begin{bmatrix} \beta_1 \\ I_{Nu} \end{bmatrix}$, N_u denotes the dimension of the control input u, N_X denotes the dimension of the state vector X, I_{Nu} represents an $N_u \times N_u$ unit matrix, I_{Nx} represents an $I_{Nx} \times I_{Nx}$ unit matrix, D_1 is the state transition matrix, and E is the incremental input matrix. The output equation can be written as

$$\eta(k) = \begin{bmatrix} I_{N_X} & 0 \end{bmatrix} \begin{bmatrix} \widetilde{X}(k) \\ \widetilde{u}(k-1) \end{bmatrix} = C\xi(k) \tag{14}$$

Let the controller's prediction horizon be N_p and the control horizon be N_c. Based on the state equation, the system state over the N_p-step prediction horizon can be expressed as

$$\begin{aligned} &\xi(k+1) = D_1\xi(k) + E\Delta\widetilde{u}(k) \\ &\xi(k+2) = D_1\xi(k+1) + E\Delta\widetilde{u}(k+1) = {D_1}^2\xi(k) + D_1E\Delta\widetilde{u}(k) + E\Delta\widetilde{u}(k+1) \\ &\cdots \\ &\xi(k+N_p) = {D_1}^{N_p}\xi(k) + {D_1}^{Np-1}E\Delta\widetilde{u}(k) + \cdots + {D_1}^0E\Delta u(k+N_p-1) \end{aligned} \tag{15}$$

Therefore, the output variables are

$$\begin{aligned} &\eta(k+1) = C\xi(k+1) = CD_1\xi(k) + CE\Delta\widetilde{u}(k) \\ &\eta(k+2) = CD_1\xi(k+1) + CE\Delta\widetilde{u}(k+1) = CD_1{}^2\xi(k) + CD_1E\Delta\widetilde{u}(k) + CE\Delta\widetilde{u}(k+1) \\ &\cdots \\ &\eta(k+N_p) = CD_1{}^{N_p}\xi(k) + CD_1{}^{Np-1}E\Delta\widetilde{u}(k) + \cdots + CD_1{}^0E\Delta u(k+N_p-1) \end{aligned} \tag{16}$$

With

$$Y_1 = \begin{bmatrix} \xi(k+1) \\ \xi(k+2) \\ \cdots \\ \xi(k+N_c) \\ \cdots \\ \xi(k+Np) \end{bmatrix}, \lambda = \begin{bmatrix} CD_1 \\ {CD_1}^2 \\ \cdots \\ {CD_1}^{N_c} \\ \cdots \\ {CD_1}^{Np} \end{bmatrix},$$

$$\psi = \begin{bmatrix} CE & 0 & 0 & \cdots & 0 \\ CAE & CE & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ {CD_1}^{N_p-1}E & {CD_1}^{N_p-2}E & {CD_1}^{N_p-3}E & \cdots & {CD_1}^{N_p-N_c}E \end{bmatrix},$$

$$\Delta U = \begin{bmatrix} \Delta\widetilde{u}(k) \\ \Delta\widetilde{u}(k+1) \\ \Delta\widetilde{u}(k+2) \\ \cdots \\ \Delta\widetilde{u}(k+N_C-1) \end{bmatrix}.$$

The predictive model of the control system is obtained through summarization as

$$Y_1 = \lambda\xi(k) + \psi\Delta U \tag{17}$$

Among these, λ is the state influence matrix, ψ is the control influence matrix, ΔU is the control increment sequence, and Y_1 is the predicted output sequence.

2.5.2. Objective Function and Constraints

In model predictive control, the system can predict the next-step state, but the control increment remains unknown. Consequently, an optimization objective must be solved at each time step to obtain an optimal control policy that satisfies the robot's motion constraints. To achieve fast and accurate trajectory tracking for the orchard inspection robot, the objective function penalizes both the state-tracking error and the control input (control increment). By taking the planned path as the reference trajectory, the MPC problem is formulated as a quadratic program. The objective function adopted in this study is given in Equation (18).

$$J(k) = \sum_{i=1}^{H_p} \hat{y}(k+i \mid k) - y_{ref}(k+i \mid k)_Q^2 + \sum_{i=0}^{H_c-1} \|\Delta\hat{u}(k+i \mid k)\|_R^2 + \rho\varepsilon^2 \tag{18}$$

The system output reference is defined as

$$Y_c = \begin{bmatrix} X_c(k+1) & X_c(k+2) & \cdots & X_c(k+N_c) & \cdots & X_c(k+N_P) \end{bmatrix} \tag{19}$$

Let $H = \psi^{\mathrm{T}} Q_Q \psi + R_R, g = E^{\mathrm{T}} Q_Q \psi$; then the cost function can be written as

$$\min_{\Delta U} J = \frac{1}{2}\Delta U^{\mathrm{T}} H \Delta U + g^{\mathrm{T}} \Delta U \tag{20}$$

Here, H is the Hessian matrix, g is the gradient vector, ΔU is the control increment vector, and the variables are optimized. Since the control inputs of the real inspection-robot system are bounded by physical limits, constraints must be imposed on the inputs when solving the optimization problem; the constraints are

$$\begin{cases} \Delta U_{\min} \le \Delta U \le \Delta U_{\max} \\ u_{\min} \le u(k) \le u_{\max} \end{cases} \tag{21}$$

where ΔU denotes the control-input increment (error) matrix, and $u(k)$ denotes the control-input matrix.

Subject to the input constraints, the evolution of the tracking error is monitored and can be expressed as

$$e(t) = \sqrt{(x_{\text{target}}(t) - x(t))^2 + (y_{\text{target}}(t) - y(t))^2} \tag{22}$$

where $(x_{\text{target}}, y_{\text{target}})$ is the desired position, (x, y) is the current actual position, and $e(t)$ is the tracking error quantity, used for real-time feedback correction.

2.5.3. Feedback Regulation Mechanism

Feedback regulation ensures system robustness. By optimizing the objective function subject to the constraints, a sequence of control increments over the control horizon is obtained, as follows:

$$\Delta U^* = \left[\Delta \widetilde{u}^*(k|k|, \Delta \widetilde{u}^*(k|k+1), \cdots \quad \Delta \widetilde{u}^*(k|k+N-1) \right] \tag{23}$$

Subsequently, the first element of the above sequence of control increments is applied as the control input to the orchard inspection robot, thereby yielding the control command at the current time, as follows:

$$u^*(k|k) = u^*(k|k-1) + \Delta u^*(k|k-1) \tag{24}$$

Among these, ΔU^* represents the unconstrained optimal solution, $u^*(k|k)$ denotes the optimal control input applied at time k, $u^*(k|k-1)$ is the actual control input applied at the previous time step, and $\Delta u^*(k|k-1)$ signifies the optimal input change at the current time step obtained through MPC optimization.

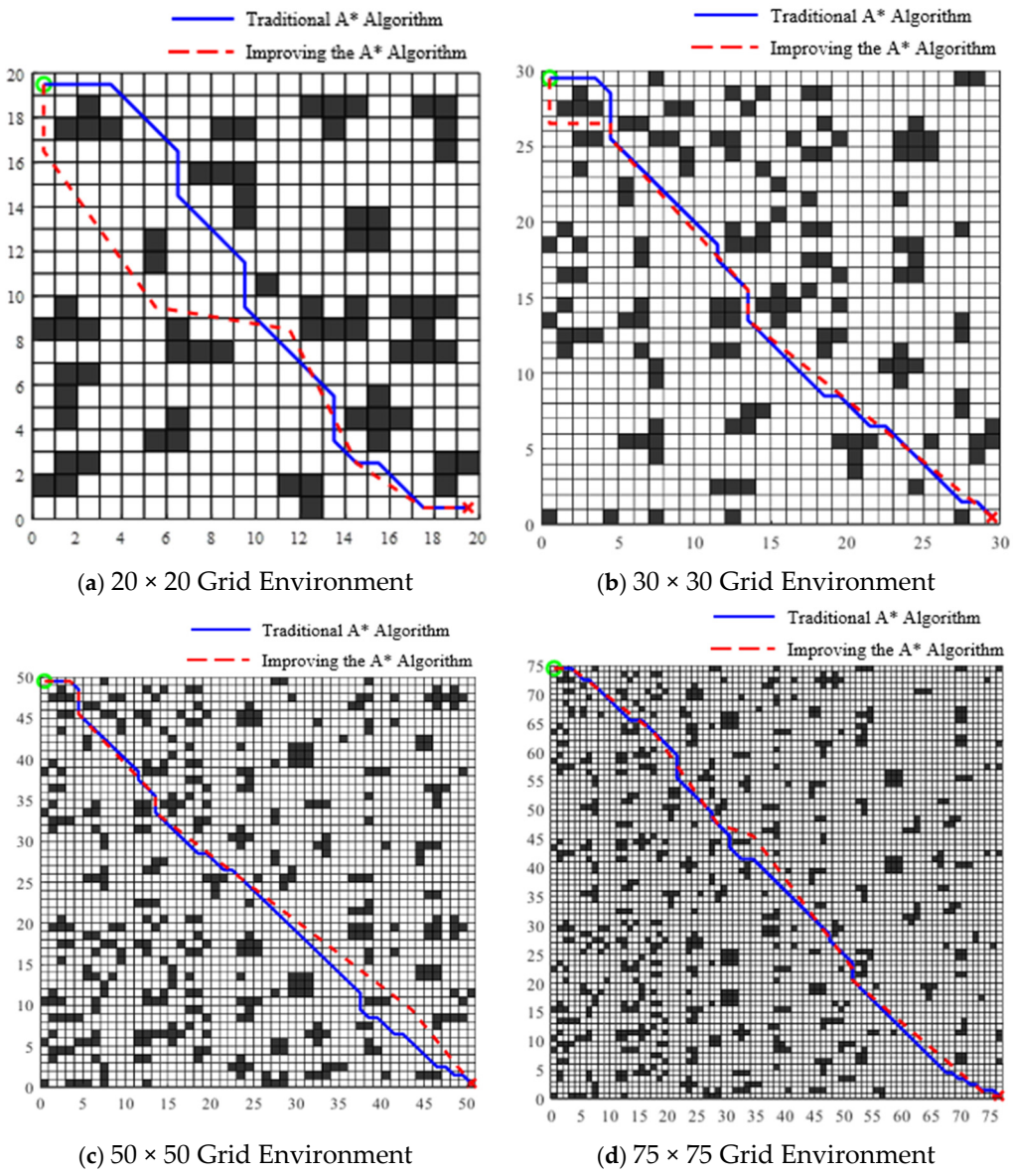
Addressing state deviations caused by localization drift and sensor noise in orchard environments, this study utilizes the MPC feedback regulation mechanism to continuously correct the predicted trajectory based on real-time sensor observations X_{act} via receding horizon optimization. This closed-loop mechanism essentially functions as a filter against uncertainties in unstructured environments: when state errors $e(t)$ arise due to ground slippage or localization jitter, the MPC automatically generates compensatory control increments Δu in the subsequent control cycle. This endows the system with intrinsic robustness without the need for explicit noise modeling, thereby successfully executing the trajectory-tracking control of the robot.

3. Results and Discussion

3.1. Simulation Results of the Path-Planning Algorithm

3.1.1. Global Path-Planning Simulations

To comprehensively evaluate the effectiveness and scalability of the improved A algorithm under varying environmental complexities, comparative experiments were conducted across four different grid map scales (20×20, 30×30, 50×50, and 75×75). To eliminate contingency and verify algorithmic robustness, no fewer than 100 independent randomized trials were performed for each map size. In each trial, the start point, goal point, and obstacle distribution were randomly generated to ensure the broad representativeness of the data. To intuitively demonstrate the specific details of path optimization, a representative trial was selected for trajectory visualization. The simulation results are illustrated in Figure 8.



(**a**) 20 × 20 Grid Environment (**b**) 30 × 30 Grid Environment

(**c**) 50 × 50 Grid Environment (**d**) 75 × 75 Grid Environment

Figure 8. Path comparison between baseline and proposed enhanced algorithms in environments of different scales.

Following the above comparative experiments, we next conduct a detailed analysis of the results with respect to key performance indicators—including planning time, path length, number of turns, number of expanded nodes, and number of path nodes—so as to more intuitively contrast the improved algorithm with the conventional algorithm. The comparative results across different grid-map environments are summarized in Table 2. The average improvement rates are calculated based on 100 independent randomized trials.

Table 2. Algorithm performance comparison.

Map Size	Algorithm	Planning Time (s)	Path Length (m)	Expanded Nodes (Count)	Path Nodes (Count)	Number of Turns (Count)	Number of Risky Nodes (Count)
20 × 20	Traditional A* algorithm	0.0867	30.38	168	26	10	20
	Proposed A* algorithm	0.0506	29.72	32.00	7.000	5.000	6.000
	Comparison (Single)	↓41.60%	↓2.17%	↓80.95%	↓73.08%	↓50%	↓70%
	Avg. Improv. rate	41.2% ± 5.9%	2.4% ± 1.2%	78.5% ± 6.4%	71.2% ± 5.5%	48.5% ± 10.2%	68.3% ± 12.4%

Table 2. *Cont.*

Map Size	Algorithm	Planning Time (s)	Path Length (m)	Expanded Nodes (Count)	Path Nodes (Count)	Number of Turns (Count)	Number of Risky Nodes (Count)
30 × 30	Traditional A* algorithm	0.1254	44.53	342.0	36.00	13.00	20.00
	Proposed A algorithm	0.0862	42.85	71.00	9.000	5.000	5.000
	Comparison (Single)	↓31.26%	↓3.77%	↓79.24%	↓75%	↓61.54%	↓75%
	Avg. Improv. rate	32.5% ± 5.1%	3.5% ± 1.4%	77.6% ± 5.8%	73.6% ± 6.1%	58.9% ± 9.8%	72.2% ± 11.3%
50 × 50	Traditional A* algorithm	0.3467	74.98	436.0	59.00	21.00	25.00
	Proposed A* algorithm	0.2013	71.69	169.0	15.00.	6.000	5.000
	Comparison (Single)	↓41.93%	↓4.39%	↓61.24%	↓74.58%	↓71.4%	↓80%
	Avg. Improv. rate	40.7% ± 7.2%	4.1% ± 1.5%	62.2% ± 6.7%	73.8% ± 5.2%	69.8% ± 8.5%	76.9% ± 9.2%
75 × 75	Traditional A* algorithm	0.5923	97.12	623.0	88.00	25.00	32.00
	Proposed A* algorithm	0.3817	92.63	256.0	19.00	7.000	10.00
	Comparison (Single)	↓35.55%	↓4.62%	↓58.91%	↓78. 41%	↓72.0%	↓68.75%
	Avg. Improv. rate	36.8% ± 6.5%	4.5% ± 1.7%	60.3% ± 7.1%	76.2% ± 5.9%	70.4% ± 7.9%	66.5% ± 10.9%

Note: A refers to the A-star pathfinding algorithm; ↓ indicates an decrease in the respective parameter.

From the above experimental data, the improved A* algorithm shows clear performance gains over the traditional A* across grid sizes. For planning time, both algorithms increase nonlinearly with map scale. The traditional A* rises from 0.0867 s on the 20 × 20 map to 0.5923 s on the 75 × 75 map, an increase of 6.8 times. The improved algorithm rises from 0.0506 s to 0.3817 s, an increase of 7.5 times, and the overall time efficiency improves by 31.26% to 41.93%. The advantage is most pronounced on the 50 × 50 map, where computation time is reduced by 41.93%. On the 75 × 75 map the gain is 35.55%, slightly lower than on medium scales but still demonstrating superiority in complex environments. The contrast in expanded nodes is the most striking. The traditional A* increases from 168 to 623 expanded nodes, revealing a heavy computational burden in large search spaces. By optimizing the heuristic, the improved algorithm keeps expansions between 32 and 256 nodes, achieving a reduction of 58.91% to 80.95%. On the 20 × 20 map the reduction reaches 80.95%, and on the 75 × 75 map it remains 58.91%, requiring only 256 expansions to obtain a high-quality path and greatly reducing memory usage and computational complexity. Regarding path quality, the improved algorithm maintains shorter paths while markedly enhancing smoothness. Path length is reduced by only 2.17% to 4.62%, yet the number of path nodes decreases by 73.08% to 78.41%, and the number of turns decreases by 50% to 72%. This indicates that the generated paths are simpler and smoother, which benefits practical motion control. The largest decrease in turns occurs on the 30 × 30 map at 61.54%. For safety, the number of risk nodes is reduced by 68.75% to 80%. On the 75 × 75 map the count drops from 32 to 10, yielding a 3.2-fold improvement in safety; on the 50 × 50 map it drops from 25 to 5, yielding a fivefold improvement. The statistical results based on over 100 randomized trials strongly corroborate the single-trial observations. The average improvement rates demonstrate consistent and significant gains: planning time reduced by 32.5–41.2%, expanded nodes reduced by 60.3–78.5%, path nodes reduced by 71.2–76.2%, turns reduced by 48.5–70.4%, and risky nodes reduced by 66.5–76.9%. The low standard deviations confirm that these performance advantages are statistically significant and highly reproducible across varying map scales. Overall, the improved A* enhances the search strategy and achieves gains in computational efficiency, path quality, and safety.

3.1.2. Fuzzy Control-Based Improved DWA Algorithm Simulation Experiments

To evaluate the performance gain of dynamic weight optimization based on fuzzy control for the DWA in complex environments, a four-level obstacle-density experimental scheme is constructed (sparse 15%, standard 30%, dense 45%, extreme 60% coverage), and comparative validation experiments are designed for scenarios with different obstacle concentrations. Using the MATLAB 2022a simulation platform, a 10 m × 10 m grid map is built with the start position set to (1, 9) and the goal position set to (10, 1). In the figures,

the blue solid line represents the path planned by the conventional DWA, and the red solid line represents the path planned by the proposed FW-DWA. The simulation results are shown in Figure 9.

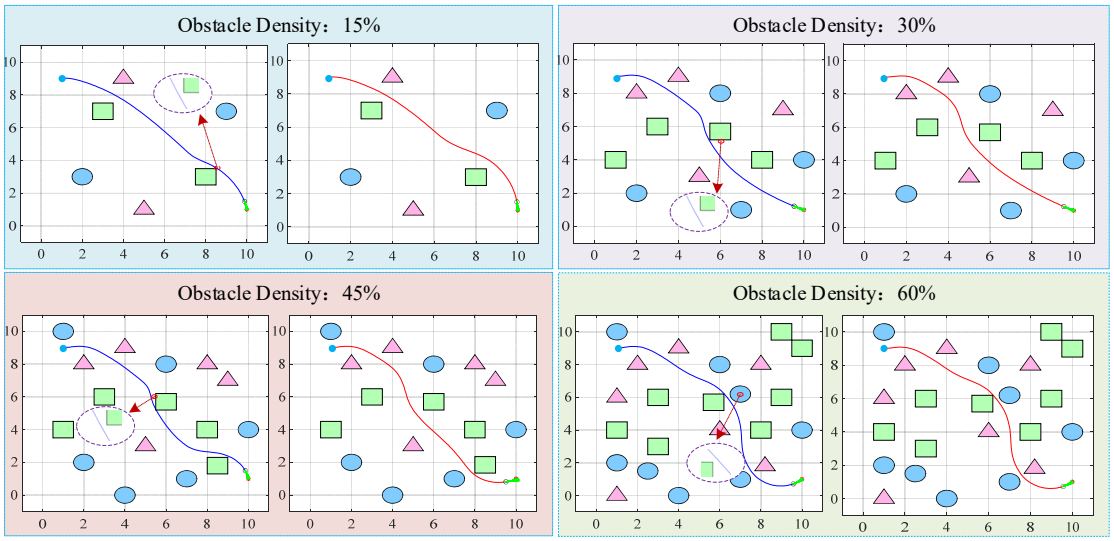


Figure 9. Comparative experiments between traditional and proposed improved DWA algorithms under different obstacle densities.

The experimental group uses a fuzzy inference mechanism to make real-time adjustments to the weight coefficients of the three evaluation functions for heading angle, obstacle distance, and speed. The control group adopts the conventional DWA with fixed weight parameters. Quantitative analysis is conducted using path length, search time, minimum safety distance, and obstacle avoidance success rate. The detailed simulation comparison is presented in Table 3.

Table 3. Simulation experimental data.

Obstacle Density	Algorithm	Path Length (m)	Search Time (s)	Minimum Safety Distance (m)
15%	Traditional DWA	12.28	52.64	0.261
	Proposed FW-DWA	11.65	54.85	0.583
	Comparison (Single)	↓5.13%	↑4.19%	↑123.3%
	Avg.change. rate	↓5.1% ± 1.2%	↑4.2% ± 1.5%	↑78.5% ± 6.4%
30%	Traditional DWA	12.91	63.15	0.253
	Proposed FW-DWA	12.35	66.83	0.4840
	Comparison (Single)	↓4.33%	↑+5.82%	↑+91.3%
	Avg.change. rate	↓4.3% ± 1.4%	↑5.8% ± 1.6%	↑91.5% ± 10.8%
45%	Traditional DWA	13.63	78.30	0.2240
	Proposed FW-DWA	12.95	83.72	0.2780
	Comparison (Single)	↓4.98%	↑6.92%	↑24.1%
	Avg.change. rate	↓5.2% ± 1.6%	↑6.9% ± 2.1%	↑24.7% ± 5.5%
60%	Traditional DWA	14.54	92.86	0.2010
	Proposed FW-DWA	13.81	98.95	0.2470
	Comparison (Single)	↓5.02%	↑6.55%	↑22.8%
	Avg.change. rate	↓5.1% ± 1.7%	↑6.4% ± 2.3%	↑23.1% ± 4.6%

Note: ↑(↓) indicates an increase (decrease) in the respective parameter.

Simulation data show that as obstacle density increases from 15% to 60%, the FW-DWA exhibits clear performance advantages over the conventional DWA. In the low-density case at 15%, the conventional DWA yields a path length of 12.28 m, a search time of 52.64 s, and a minimum safety distance of 0.261 m. FW-DWA reduces the path length to 11.65 m, a decrease of 5.1%, with a slight increase in search time to 54.85 s, while raising the safety distance to 0.583 m, which is 2.23 times that of the conventional method. At 30% density, the conventional DWA results are 12.91 m, 63.15 s, and 0.253 m. FW-DWA achieves 12.35 m, 66.83 s, and 0.484 m, nearly twice the safety distance, effectively balancing path efficiency and avoidance safety. At 45% density, the conventional DWA gives 13.63 m, 78.30 s, and 0.224 m. FW-DWA yields 12.95 m (about 5% shorter), 83.72 s, and 0.278 m, which is 24.1% higher in safety distance. Under the extreme 60% density, the conventional DWA reaches 14.54 m, 92.86 s, and 0.201 m. FW-DWA attains 13.81 m, 98.95 s, and 0.247 m, an increase of 22.9% in safety distance. Statistical analysis from over 100 randomized trials confirms the algorithm's robustness across all densities. Path length shows a stable average reduction of 4.3–5.2%, while search time increases only slightly (4.2–6.9%), representing an acceptable trade-off for enhanced performance. Crucially, safety margins improve significantly, increasing by 78.5–91.5% at lower densities and maintaining a robust 23.1–24.7% improvement even in highly constrained, high-density environments. These low-variance results verify the method's reliability and consistent safety advantages.

In summary, by introducing a fuzzy-logic control mechanism, FW-DWA consistently outperforms the conventional DWA across obstacle densities, with pronounced advantages in path optimization and safe obstacle avoidance, making it well suited to complex and variable real-world navigation scenarios.

3.2. Tracking Control Simulation Results

3.2.1. Straight-Line Tracking Experimental Results

Three trajectory-tracking experiments were conducted using straight-line, U-shaped, and circular paths. In all experiments, the reference angular velocities of the left and right tracks were 10 rad/s, the sampling period was 0.1 s, and the simulation duration was 1.2 s. The MPC adopted a prediction horizon of 60 and a control horizon of 20 to ensure accurate tracking of the designated path. The selection of = 60 accounts for the significant inertia and response delay of the tracked vehicle, enabling the controller to anticipate future path curvature and reduce overshoot. The control horizon = 20 was determined via empirical tuning to balance control performance against computational load, ensuring the optimization problem remains solvable within the 0.1 s cycle on the embedded hardware. The tracking results are shown in Figure 10.

In the straight-line trajectory-tracking experiment for the orchard inspection robot, three trajectories starting from different initial positions showed strong control performance and rapidly merged onto the target line, clearly reflecting the system's adaptability and stability in reference-path following. According to the tracking plots, the controller was initialized at (0, 2), (2, 2), and (3, 2), and effective dynamic adjustment was observed during motion. Trajectory 1 coincided with the reference at x of about 2.7 m; trajectory 2 started at (2, 2) directly on the reference; trajectory 3 started at (3, 2) and merged at x of about 3.69 m, yielding a compact path.

The position-error curves in Figure 10b indicate that trajectory 1 had an initial deviation of about 1.4 m, dropped below 0.1 m within 2.4 s, and fell to within 50 mm after 2.5 s; the peak deviations of trajectories 2 and 3 were 0 m and about 0.71 m, respectively, and both decayed to below 0.02 m within 3.2 s, with overall convergence exceeding 96%.

For heading error, the maximum positive error of trajectory 3 was about +46.98°, and the maximum negative error of trajectory 1 was about −72.67°; by 4s all errors were within

±2.15°, as shown in Figure 10c, demonstrating strong attitude-holding capability. The linear-velocity curves show start-up peaks of about 1.7 m/s for trajectory 1 and 0.7 m/s and 1.3 m/s for trajectories 2 and 3, and after 4 s the speed stabilized at (0.7 ± 0.05) m/s with fluctuations within 5%, indicating precise speed control under dynamic conditions (Fiure 10−d). The angular-velocity curve (Figure 10e) shows a maximum pulse of about ±1.15 rad/s and rapid settling near 0.05 rad/s, reflecting good angular stability. The left- and right-track angular-velocity profiles (Figure 10f) exhibit dynamic balance, oscillating around 7 rad/s and ensuring smooth and accurate motion along the straight trajectory.

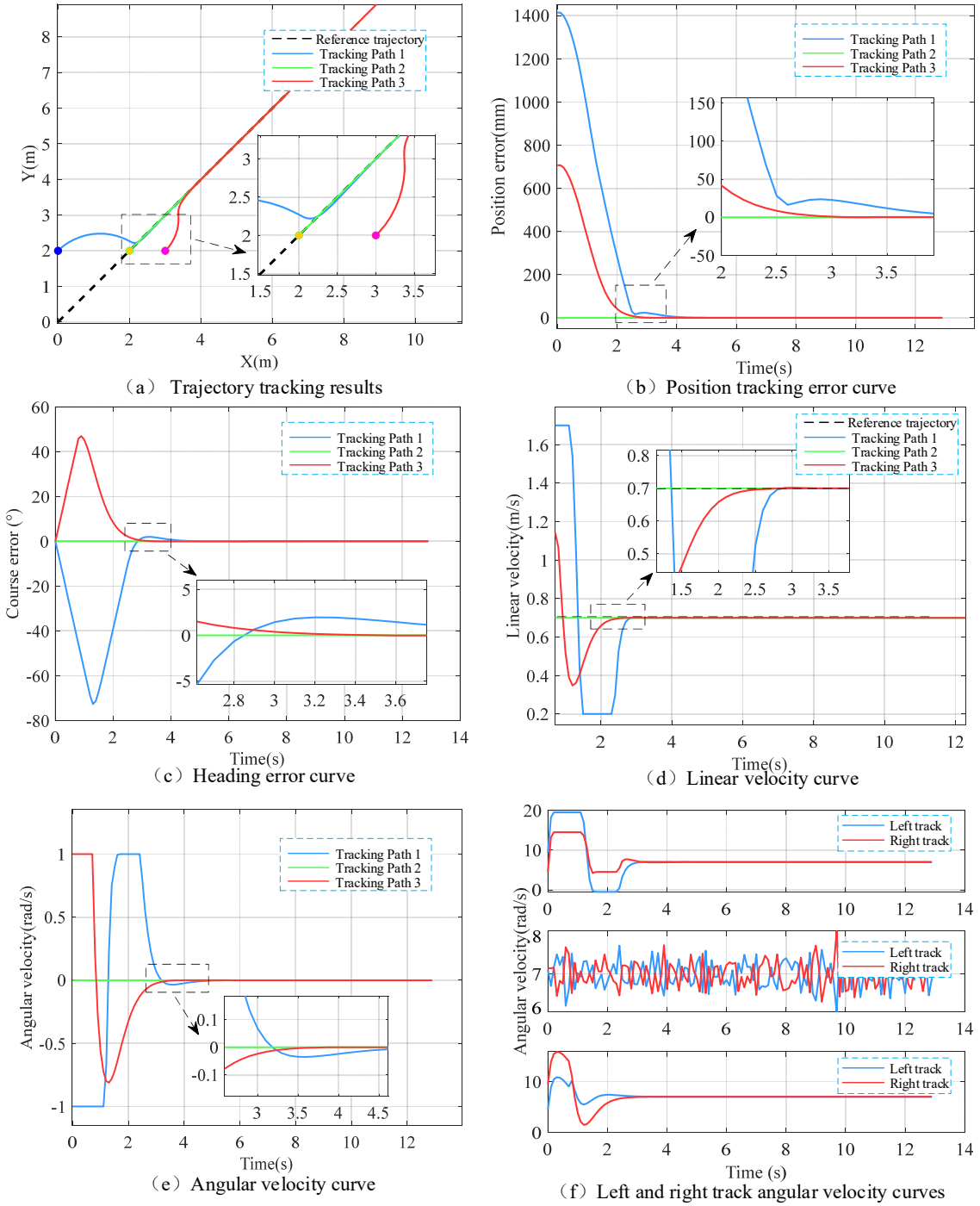


Figure 10. Straight-line trajectory-tracking experiment of the orchard inspection robot.

3.2.2. U-Shaped Curve Tracking Experimental Results

In the U-shaped trajectory-tracking experiment for the orchard inspection robot, three trajectories with different starting positions demonstrated the controller's tracking performance and robustness on a complex path. As shown in Figure 11a, the reference trajectory has a typical U shape comprising a straight segment, two 90° turns, and a return leg, which places higher demands on dynamic response. Trajectory 1 starts at (1, 0.5), trajectory 2 at (2, 1), and trajectory 3 at (1.5, 1.2), exhibiting distinct merging behaviors in the initial stage. Trajectories 1 and 3 approach the reference from opposite sides and essentially converge at x about 3.2 m; trajectory 2, being closer to the reference line at the start, shows more direct tracking. In the U-turn region (x about 7–8 m), all three trajectories closely follow the curvature of the reference path, indicating excellent adaptability to nonlinear paths.

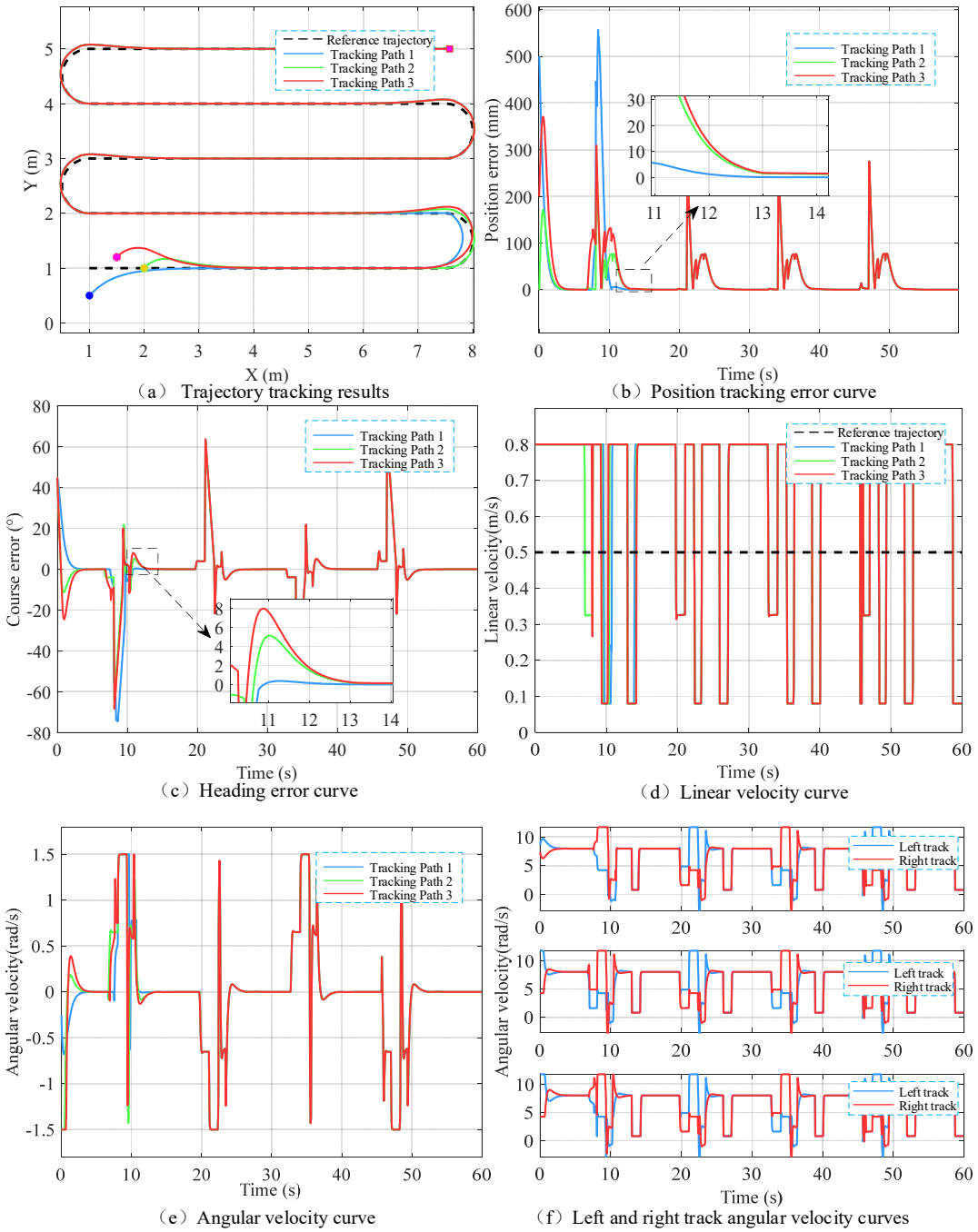


Figure 11. U-shaped curve trajectory-tracking experiment of the orchard inspection robot.

The position-tracking error curves in Figure 11b show that trajectory 1 attains a peak deviation of about 556.7 mm, mainly at the first turn, and then rapidly converges to within 50 mm in 2.5 s; the peak deviations of trajectories 2 and 3 are about 96.8 mm and 309.3 mm, respectively, both decaying to below 0.05 m within 3 s. A small rise appears at the U-turn, but the peaks remain within 280 mm. The enlarged view indicates a steady-state tracking accuracy of ± 5 mm, and the overall convergence rate exceeds 95%.

For heading error (Figure 11c), the maximum positive error of trajectory 1 is about $+63.7^\circ$ during the initial adjustment, and the maximum negative error of trajectory 3 is about -74.4° at the first turn. After the turns, the heading error is compressed to within $\pm 5^\circ$ in 5 s; the zoomed view shows that on straight segments it remains within $\pm 2^\circ$, demonstrating fast and accurate attitude control.

The linear-velocity curves (Figure 11d) indicate start-up peaks of 0.8 m/s, 0.75 m/s, and 0.8 m/s for trajectories 1, 2, and 3, adaptive deceleration to below 0.4 m/s in turning segments, and stabilization at 0.8 ± 0.03 m/s on straight segments, evidencing reasonable speed planning. The angular-velocity curve (Figure 11e) shows a maximum pulse of about ± 1.5 rad/s and rapid settling near ± 0.05 rad/s on straight segments, yielding swift and smooth angular response. The left and right track angular-velocity profiles (Figure 11f) exhibit differential-steering characteristics: in turns the difference between the left and right tracks is about 11.7 rad/s, while on straight segments both tracks remain nearly synchronized around 8 rad/s, ensuring flexibility and stability along the complex curved trajectory.

3.2.3. Circular Curve Tracking Experimental Results

In the circular trajectory-tracking experiment for the orchard inspection robot, three trajectories starting from different initial positions all converged to the target circular path. From the tracking plots, trajectory 1 starts at (0, 2), trajectories 2 and 3 start at (2, 2) and (3, 2), and all ultimately track the reference circle with a radius of 4.5 m with high accuracy. The position-tracking error curves show initial errors of about 4500 mm, 2500 mm, and 1500 mm for trajectories 1, 2, and 3, respectively, with trajectory 1 exhibiting the largest initial deviation because its starting point coincides with the circle center. After about 4.7 s of adjustment, the position errors for all trajectories rapidly converge to within 0.12 m, with a convergence rate exceeding 97%. For heading error (Figure 12c), the peak values for the three paths are -70.78°, -62.19°, and -52.29°, and by 5.1 s they are within $\pm 2^\circ$. During steady circular motion they remain within 2°, yielding a heading-control accuracy above 95%. The linear-velocity curves (Figure 12d) indicate acceleration peaks of about 3.3 m/s, followed by two underdamped oscillations, and convergence within 6s to the desired value of 2.3 ± 0.1 m/s with fluctuations not exceeding 5%. The angular-velocity curves (Figure 12e) show a maximum peak of about ± 1.2 rad/s, followed by stabilization around 0.5 rad/s. The left- and right-track angular-velocity profiles (Figure 12f) exhibit actuator saturation: the start segment peaks at about 30 rad/s; the mid segment decreases with linear speed to about 18 rad/s while remaining symmetric, and from 9 s it rises again to about 22 rad/s to complete the circular loop.

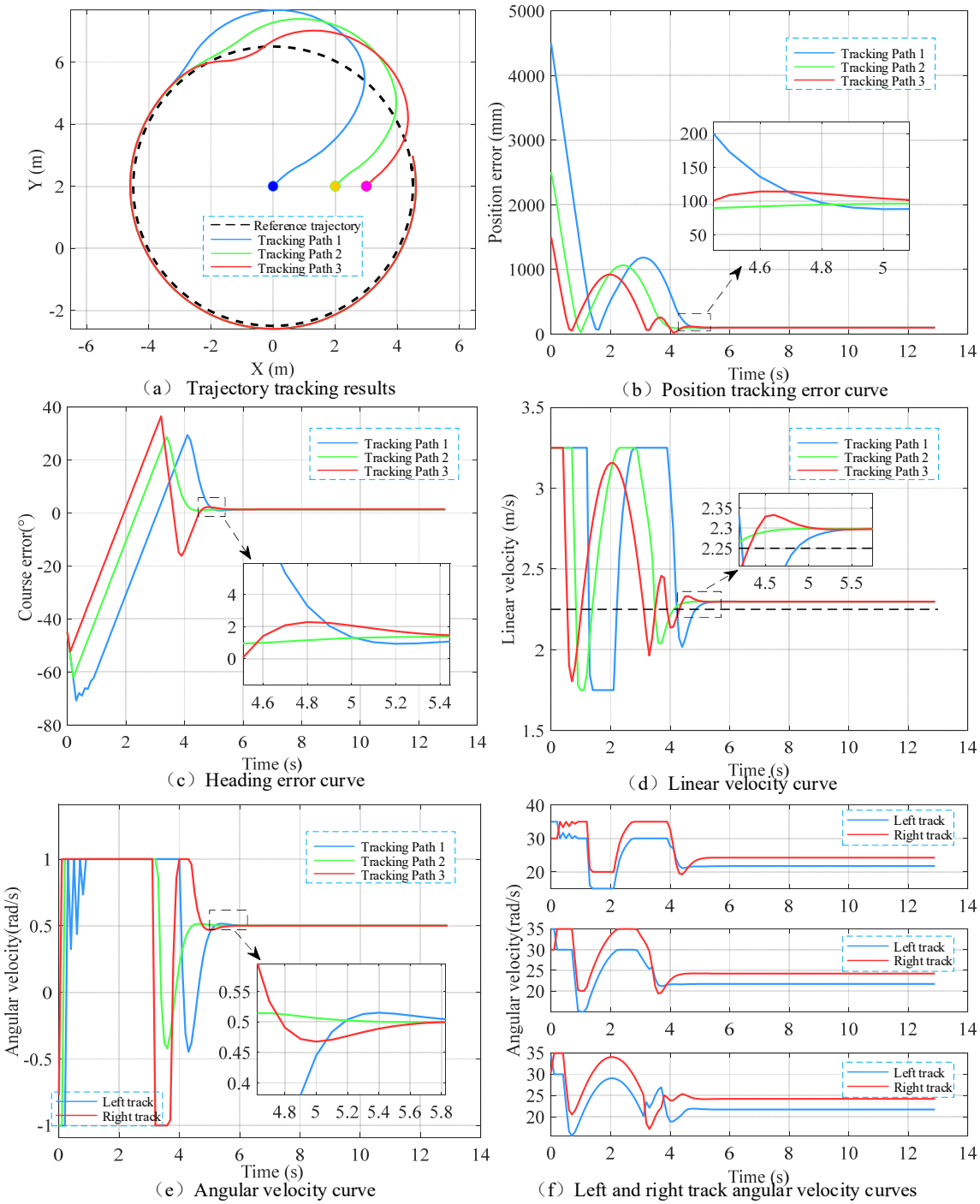


Figure 12. Circular trajectory-tracking experiment of the orchard inspection robot.

3.3. Field Experiments

3.3.1. Scenario Design

To verify the practical effectiveness of the proposed path-planning and trajectory-tracking algorithms, experiments were conducted in two settings: the Robotics Laboratory at Chongqing University of Science and Technology and a real orchard environment. The field experiment phase of this study aims to expose the system to various uncertainties inherent in real agricultural environments, including track slippage and kinematic model mismatches caused by uneven terrain, LiDAR point cloud jitter resulting from robot vibration, and execution deviations such as motor response lag. These uncertainties are intrinsic to actual operating environments and realistically reflect the challenges encountered in

engineering applications. Therefore, by conducting experimental validation under non-ideal conditions, this study evaluates the stability and reliability of the proposed method in coping with these uncertainties. Furthermore, the online perception and obstacle avoidance processes primarily rely on real-time environmental data acquired by LiDAR to ensure the accuracy and robustness of path planning and avoidance.

(1) Indoor experimental scenario

The indoor simulation and verification of path planning and trajectory tracking were carried out in the Robotics Laboratory at Chongqing University of Science and Technology.

The inspection robot prototype is compact, measuring 0.26 m in length, 0.206 m in width, and 0.193 m in height, and it adopts a tracked drive configuration. A scaled mock orchard measuring 5 m × 5 m was constructed on the lawn outside the laboratory. Small potted plants with a diameter of 0.2 m were used to emulate the distribution of fruit trees, arranged with 1.5 m plant spacing and 1.5 m row spacing, maintaining a 1:3 scale relative to a real orchard, as shown in Figure 13.

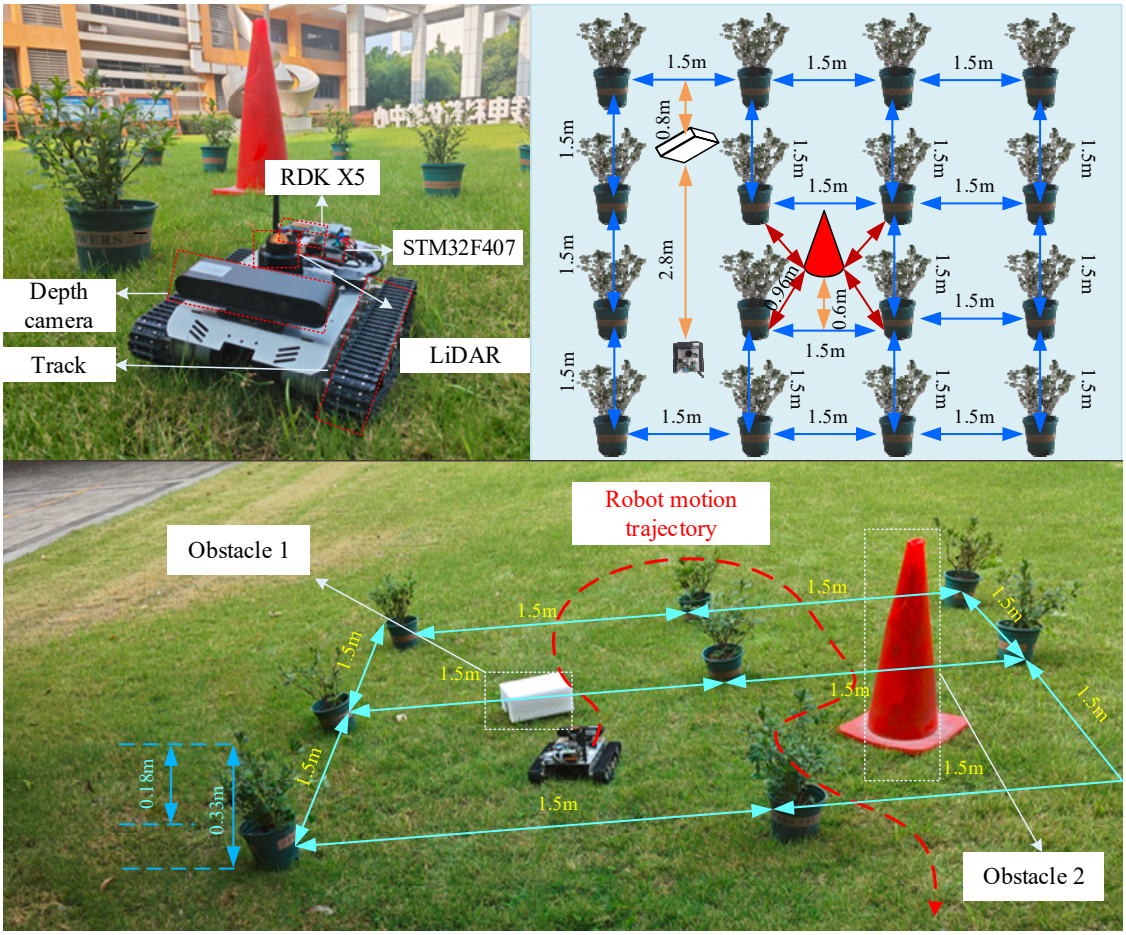


Figure 13. Schematic diagram of the lawn simulation experiment scene.

(2) Real-orchard experimental scenario

The field validation was conducted at an orchard planting experimental base in Shapingba District, Chongqing, located at 106°19′ E, 29°35′ N. The orchard adopts a wide-row dense planting scheme, with single-tree spacing of 2.8 m and aisle width of 2.2 m to facilitate mechanized operations. Mature citrus trees have an average height of 2.3 m, a canopy spread of 1.8–2.2 m, and moderately dense foliage. The site contains five main lanes with a width of 1.5 m and branch lanes with a width of 0.8 m, forming a grid traffic network. The test area is a rectangular plot measuring 12 m in length and 8 m in width; the

southwest corner is set as the origin of the coordinate system to establish a local reference frame. The inspection robot was tested in the real orchard, as shown in Figures 14 and 15.

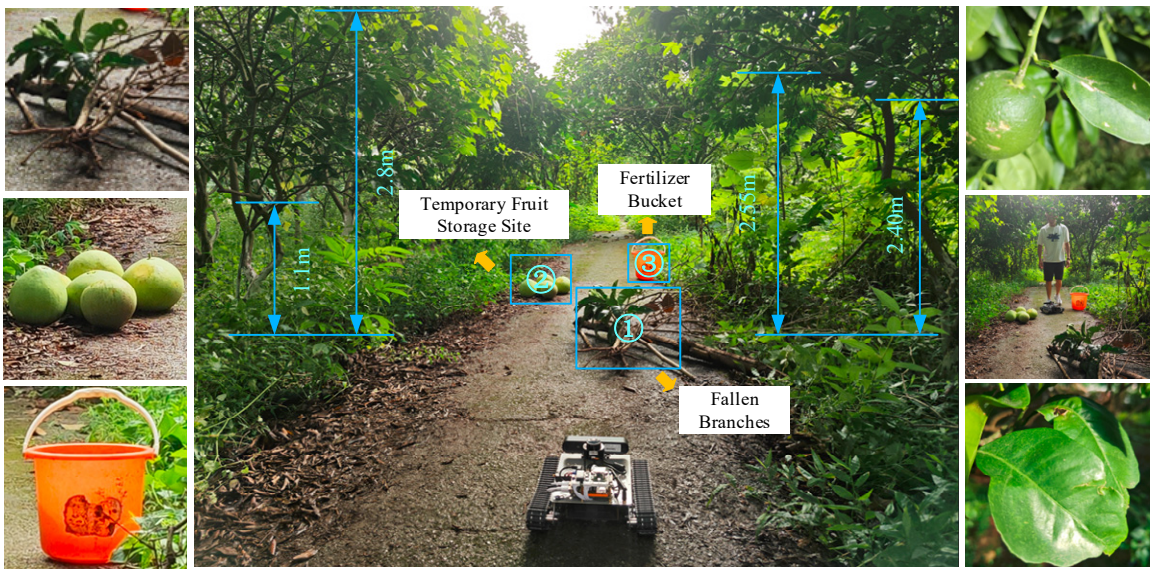


Figure 14. Schematic diagram of the real orchard experimental scene.

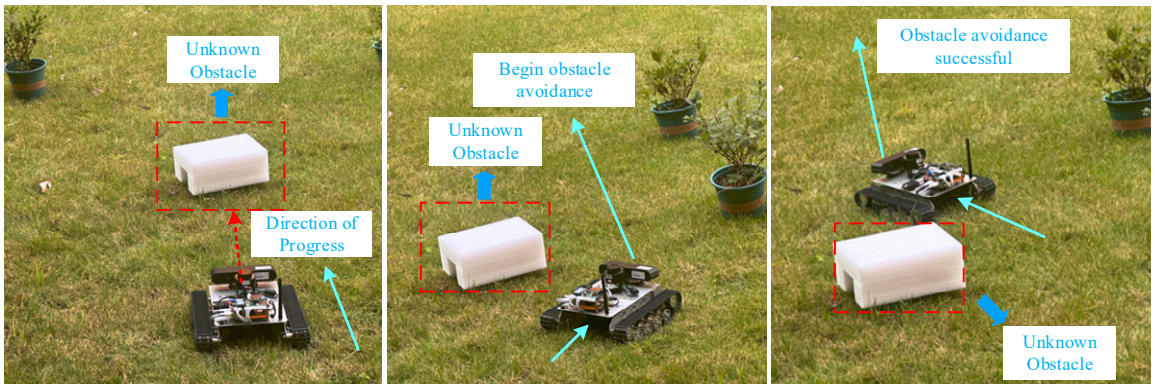


Figure 15. Schematic diagram of obstacle avoidance.

In the experimental scenario, the start and goal of the inspection robot are first specified, and the path-planning algorithm generates an optimal inspection path based on these parameters. The robot then proceeds along the planned route while acquiring environmental data and state information in real time to ensure accurate task execution. During the experiment, the robot follows the prescribed path and makes dynamic adjustments when necessary to avoid obstacles. The entire process is analyzed comprehensively using charts and data logs to verify the effectiveness and robustness of the proposed algorithm under different environmental conditions.

3.3.2. Experimental Results

In this experiment, the robot performs an inspection task in an environment containing moving obstacles. Using real-time sensor data, it detects the positions and motion trajectories of dynamic obstacles and adjusts its travel path according to the prescribed algorithm to ensure safe and effective avoidance. During the experiment, the robot's response time, path length, and minimum safety distance are recorded to assess the algorithm's performance and reliability. The dynamic obstacle-avoidance procedures are shown in Figures 16 and 17. The experimental improvement results are shown in Table 4.

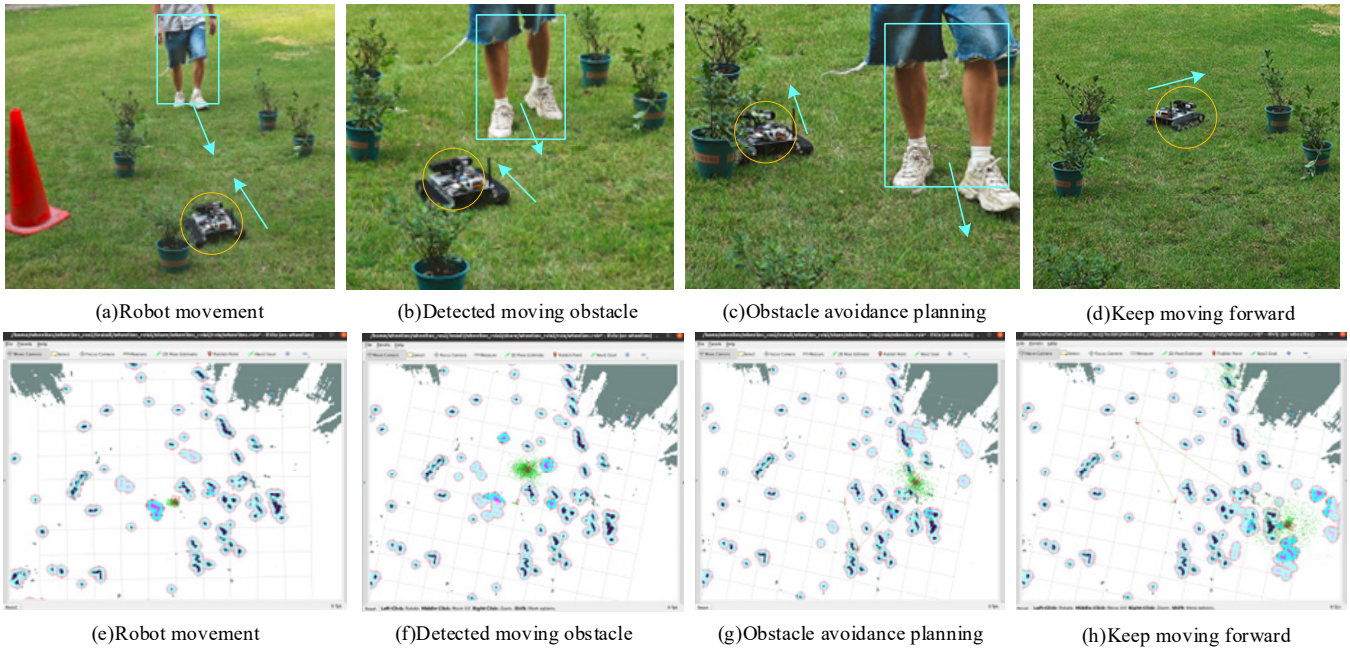


Figure 16. Path-planning experiment in the lawn environment.

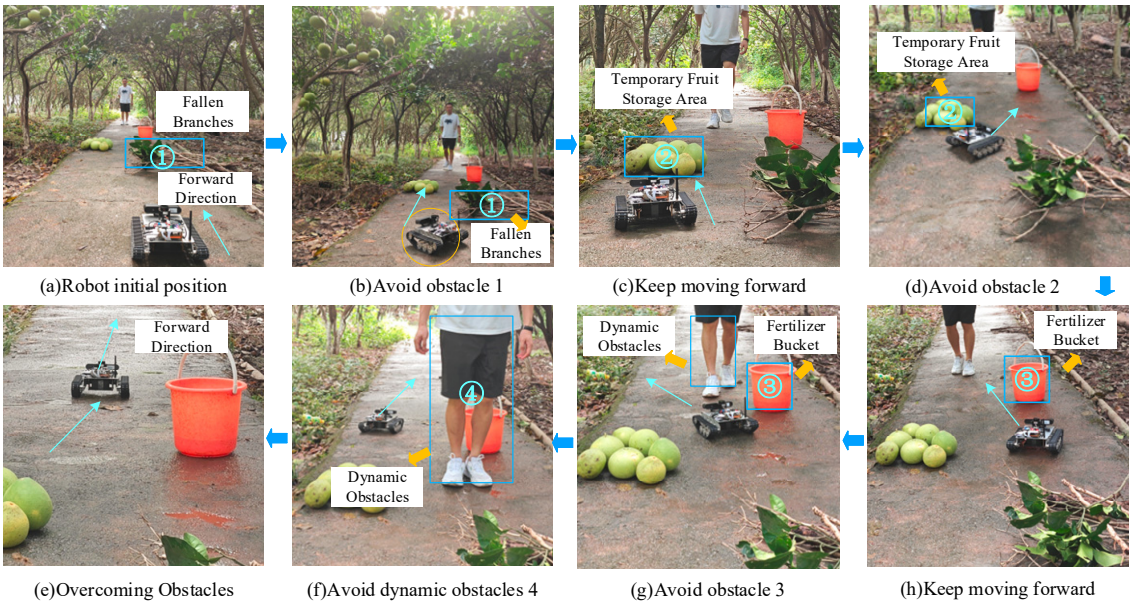


Figure 17. Path-planning experiment in the orchard environment.

Table 4. Comparison of core metrics before and after algorithm improvement.

Environment	Indicator	Traditional A* and DWA	Improved A* and DWA	Improv. Rate (Single)	Improv. Rate (Mean ± SD)
Simulation	Path Length (m)	15.88	14.75	↓7.12%	↓7.08% ± 1.51%
	Time (s)	42.26	39.74	↓5.96%	↓6.12% ± 2.83%
	Safety distance (m)	0.2600	0.3200	↑23.08%	↑22.71% ± 5.55%
Orchard	Path Length (m)	13.65	12.86	↓5.79%	↓5.8% ± 1.82%
	Time (s)	51.73	49.85	↓3.63%	↓3.59% ± 3.17%
	Safety distance (m)	0.1800	0.2700	↑50.01%	↑49.88% ± 8.21%

Note: A refers to the A-star pathfinding algorithm; ↑ (↓) indicates an increase (decrease) in the respective parameter.

To ensure reliability and repeatability, each control strategy in each scenario was executed under identical conditions and repeated. To comprehensively assess the performance

gains of the improved method, comparative experiments were conducted on key metrics for two algorithmic combinations, traditional A* with the DWA, and improved A* with the FW-DWA, in both simulated and real-orchard environments. The detailed results are listed in the table. In the simulated environment, the improved method reduced path length from 15.88 m to 14.75 m, a shortening of 7.12%, indicating that the improved A* effectively lowers path redundancy through optimized node expansion. Planning time decreased from 42.26 s to 39.74 s, an efficiency gain of about 5.96%, attributable to the adaptive mechanism in FW-DWA that reduces unnecessary trajectory evaluations. The minimum safety distance increased from 0.26 m to 0.32 m, an improvement of 23.08%, showing that the improved method reserves a larger safety margin and lowers collision risk.

In the real-orchard tests, the advantages are more pronounced. Path length fell from 13.65 m to 12.86 m, an optimization of 5.79%, demonstrating better adaptation to narrow inter-row passages and generation of more compact routes. Planning time was compressed from 51.73 s to 49.85 s, an efficiency gain of 3.64%, indicating stable computational performance despite higher environmental complexity. The minimum safety distance rose from 0.18 m to 0.27 m, an increase of 50%, which is critical in orchards with densely distributed trees and trellis structures. Across both environments, the improved A* plus FW-DWA achieved significant gains in path quality, computational efficiency, and safety: average path length shortened by 6.46%, with time savings of 4.80%, and safety distance increased by 36.54%, confirming the practical value and technical superiority of the proposed scheme for orchard inspection path planning. Statistical results from more than 15 independent and valid trials further substantiate these findings. In the simulation environment, path length is reduced by an average of 7.08% $\pm$ 1.51%, planning time decreases by 6.12% $\pm$ 2.83%, and safety distance improves by 22.71% $\pm$ 5.55%. In the real-orchard environment, the average improvements are 5.8% $\pm$ 1.82% for path length, 3.59% $\pm$ 3.17% for planning time, and 49.88% $\pm$ 8.21% for safety distance. The close agreement between single-trial and mean values, combined with low standard deviations, confirms the statistical significance, reproducibility, and robustness of the proposed algorithm across diverse operational scenarios.

To validate the effectiveness and stability of the MPC-based trajectory-tracking algorithm, a complete comparative protocol was designed. Experiments were repeated multiple times under identical conditions in the same scenarios, and each run was recorded and fitted for analysis, enabling a comprehensive assessment across operating conditions. Using the reference paths generated by the path-planning algorithms, trajectory-tracking comparisons were performed with MPC and PID control. Systematic analysis of the repeated trials enables an objective evaluation of tracking accuracy, stability, and robustness for the different control methods. The comparative error experiment results are shown in Tables 5 and 6.

Table 5. Experimental results of position errors for different control algorithms.

Control Algorithm	Overall Maximum Error (m)	Overall Average Error (m)	Maximum Error in Straight Segments (m)	Average Error in Straight Segments (m)	Maximum Error in Arc Segments (m)	Average Error in Arc Segments (m)
PID	0.0970	0.0520	0.0780	0.0490	0.0970	0.0570
MPC	0.0510	0.0320	0.0410	0.0310	0.0510	0.0330

Table 6. Experimental results of heading errors for different control algorithms.

Control Algorithm	Overall Maximum Error (°)	Overall Average Error (°)	Maximum Error in Straight Segments (°)	Average Error in Straight Segments (°)	Maximum Error in Arc Segments (°)	Average Error in Arc Segments (°)
PID	33.15°	6.400°	20.19°	4.470°	33.15°	9.750°
MPC	16.31°	3.140°	10.38°	2.490°	16.31°	5.420°

The experimental results are presented in Figure 18 as fitted curves. The figure contains four subplots, each showing a different performance dimension: Figure 18a shows the robot's actual trajectory during the experiment, providing an intuitive view of overall path-tracking performance under the two control methods; Figure 18b shows the time history of position-tracking error, which quantifies the deviation between the robot and the preset path and enables a direct comparison of the tracking accuracy of MPC and PID; Figure 18c shows the time history of heading error, which evaluates the precision of directional control for the different strategies; and Figure 18d shows the velocity time-series, which reflects differences in the motion-control dynamics of the two methods. A comprehensive analysis of these multi-dimensional data enables a full assessment of MPC and PID in the trajectory-tracking task for the tracked inspection robot and provides experimental evidence for controller optimization.

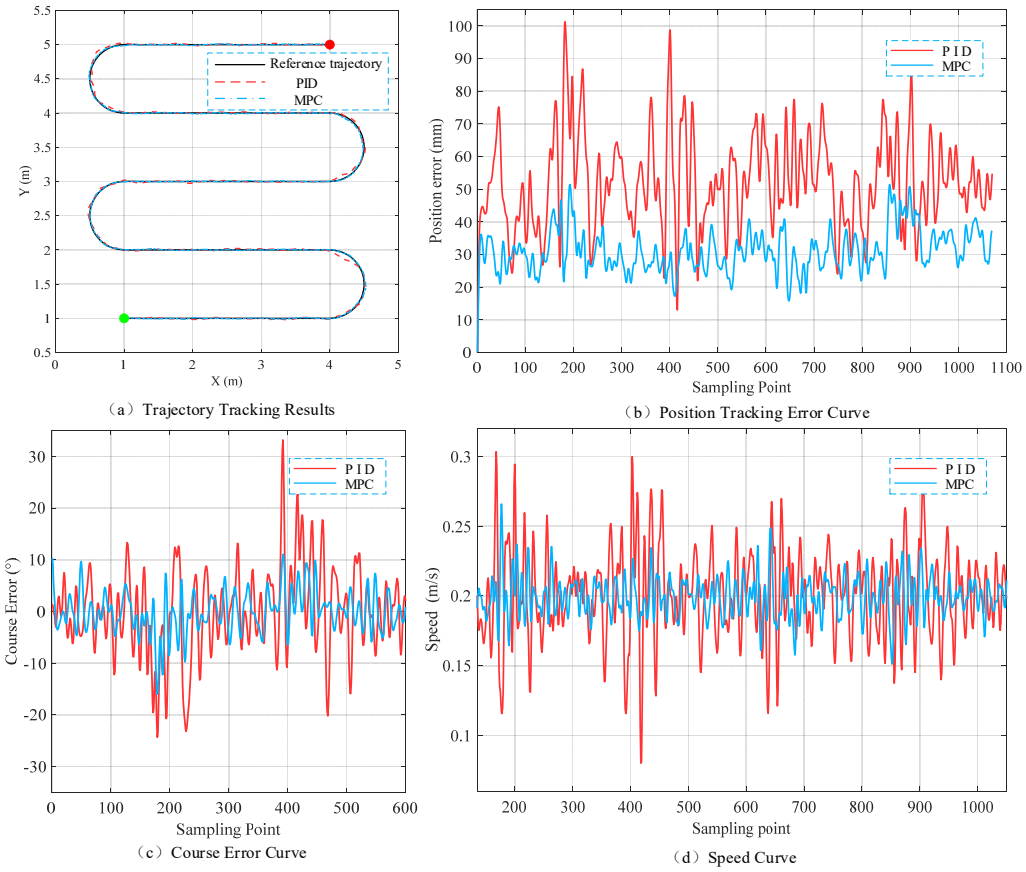


Figure 18. Comparison of tracking errors between MPC and PID.

To evaluate the algorithm's performance under actual uncertainties, this study compared ideal simulations with field orchard test data and conducted comparative experiments between MPC and PID controllers on a standardized test path. Both position and heading errors were analyzed. The results indicate that MPC significantly outperforms PID in tracking accuracy. Regarding position error, MPC achieved a maximum error of 0.051 m and a mean error of 0.032 m, representing reductions of 46.39% and 40.38% compared to PID's 0.097 m and 0.052 m, respectively. On straight segments, MPC yielded a maximum error of 0.041 m and a mean error of 0.031 m, improving upon PID's 0.078 m and 0.049 m by 47.44% and 36.73%, respectively. On curved segments, MPC recorded a maximum error of 0.051 m and a mean error of 0.033 m, reflecting improvements of 47.42% and 42.11% over PID's 0.097 m and 0.057 m. Although track slippage caused by ground undulations and environmental noise in LiDAR data led to minor local trajectory oscillations, the

system exhibited neither divergence nor collisions. These results demonstrate that MPC, through its prediction and receding horizon optimization mechanisms, achieves higher precision trajectory tracking on both linear and nonlinear paths, effectively suppressing trajectory deviation.

The analysis of heading error further corroborates the superiority of MPC. Overall, MPC recorded a maximum heading deviation of 16.31° and a mean of 3.14°, compared to 33.15° and 6.40° for PID. This corresponds to reductions of 50.80% in peak error and 50.94% in mean error, indicating MPC's stronger capability to suppress deviations under steady-state conditions. On straight segments, MPC achieved a maximum deviation of 10.38° and a mean of 2.49°; compared to PID's 20.19° and 4.47°, these represent reductions of 48.60% and 44.30%, respectively. On curved segments, MPC showed a maximum deviation of 16.31° and a mean of 5.42°; relative to PID's 33.15° and 9.75°, the mean error improved by 44.41%.

Combining the trajectory overlay comparisons in Figure 18a–d, and acknowledging unavoidable disturbances such as ground undulations and system lag during field testing, the tracking error reflects not only control accuracy but also the strategy's ability to suppress uncertain disturbances. MPC outperformed PID in both mean position error (0.032 m) and mean heading error (3.14°), demonstrating that its receding horizon optimization and feedback correction enable more stable trajectory-tracking performance under non-ideal conditions. Particularly in complex dynamic environments where PID exhibited larger tracking errors, MPC effectively mitigated errors arising from ground disturbances and sensor noise, exhibiting superior disturbance rejection capabilities.

3.3.3. Engineering Deployment Constraints and Real-Time Performance Analysis

To clarify the engineering feasibility of the proposed system, this section provides a quantitative analysis and testing of computational resource consumption, parameter calibration requirements, and system reliability on the target hardware platform.

(1) Analysis of Computational Performance

Test results based on the RDK X5 platform, as shown in Table 7, indicate that the system's computational load remains within a highly efficient and controllable range. The collaborative operation of the improved A*, FW-DWA, and MPC modules stabilizes the overall navigation control cycle within 500 ms, satisfying the real-time obstacle avoidance requirements of narrow orchard environments.

Table 7. Computational resource consumption of each algorithm module.

Algorithm Module	CPU Usage	Execution Time per Cycle
Improved A* Global Planning	35–45%	120 ms
FW-DWA Local Planning	20–28%	45 ms
MPC Trajectory Tracking	15–22%	18–25 ms
System Total	70–85%	<500 ms (Complete Cycle)

(2) Parameter Calibration Strategy

To reduce engineering deployment complexity, the system employs a decoupled hierarchical calibration strategy. Only seven core parameters associated with the planning and control layers require offline calibration based on typical scenarios. Experimental validation confirms that this configuration exhibits excellent transferability across both simulated and real-world environments, obviating the need for repeated on-site tuning. This significantly lowers the labor costs and technical barriers for practical application. The calibration specifications for the system's core parameters are presented in Table 8.

Table 8. Calibration description of core system parameters.

Parameter Layer	Parameter Name	Symbol	Range	Calibration Method
Global Planning Layer	Heuristic adjustment factor	β	0.6–0.8	Offline simulation optimization
Local Planning Layer	Target distance domain	D	[0, 6] m	Set based on orchard dimensions
Local Planning Layer	Obstacle distance domain	O	[0, 2] m	Set based on aisle width
Trajectory Tracking Layer	Prediction horizon	N_p	50–70	Balance accuracy vs. computation
Trajectory Tracking Layer	Control horizon	N_c	15–25	Balance response vs. stability
Trajectory Tracking Layer	State weighting matrix	Q	diag [1, 1, 0.5]	Set based on tracking priorities
Trajectory Tracking Layer	Control weighting matrix	R	diag [0.1, 0.1]	Set based on actuator characteristics

(3) System Reliability and Fault Tolerance Mechanism

To ensure operational reliability in harsh orchard environments, the system implements a multi-layered protection strategy. At the sensor level, the data link is monitored in real time; if the packet loss rate exceeds a threshold or data frame anomalies are detected, an alarm is triggered, and the system automatically initiates deceleration or enters a standby protection mode. At the execution level, a watchdog mechanism monitors the communication between the high-level and low-level controllers. If no valid control command is received within a 2 s window, emergency braking is automatically executed. Continuous 3 h field testing verified that this mechanism effectively prevents risks associated with communication interruptions and loss of control, ensuring the stable operation of the system. Field experiments in Section 3.3.2 demonstrate that the FW-DWA algorithm significantly enhances safety during low-speed obstacle avoidance by dynamically adjusting weights via fuzzy inference. However, constrained by the sensor refresh rate, the system exhibits a certain latency when responding to high-speed dynamic targets. Furthermore, the robot possesses a continuous operational capability of approximately 4 h. Notably, the MPC Controller effectively compensates for variations in motor characteristics caused by battery depletion through receding horizon optimization, thereby ensuring the robustness of trajectory tracking during prolonged operations.

4. Conclusions

To address the complexity and constraints associated with the autonomous navigation of inspection robots in semi-structured orchard environments, this paper proposes a hierarchical collaborative navigation framework centered on the consistent trade-off between safety and efficiency. This framework aims to achieve a unified design for global planning, local obstacle avoidance, and constrained tracking. Specifically, the global planning layer optimizes search behavior by dynamically adjusting heuristic weights and incorporating risk node elimination with redundant node pruning, thereby generating safer, smoother, and more trackable reference paths. The local obstacle avoidance layer utilizes a fuzzy inference method to update the evaluation function weights of the dynamic window in real time, maintaining path feasibility and safety margins in environments characterized by dynamic obstacles and narrow passages. At the execution layer, MPC is employed for receding horizon optimization to ensure high-precision tracking of reference trajectories subject to kinematic and input constraints, systematically enhancing navigation reliability and engineering applicability in orchard scenarios.

Experimental results show that the improved A* algorithm reduces the number of expanded nodes by 58.91% to 80.95% in different-scale environments, improves planning efficiency by 31.26% to 41.93%, and reduces risk nodes by 68.75% to 80%. The fuzzy-weighted DWA algorithm optimizes the path length by 5.1%, increases the safety distance to 2.23 times that of the traditional algorithm, and significantly enhances dynamic obstacle avoidance capabilities. The overall average position error of the MPC trajectory-tracking controller is 0.031 m, a 40.38% reduction compared to PID, with an average heading error of 3.14°, showing a 50.94% improvement, fully validating its superiority in complex trajectory-tracking tasks.

Field tests in the orchard further demonstrate the practicality of the system. In the real operational environment, the path length is optimized by 5.79%, planning time is reduced by 3.64%, and the safety distance is improved by 50%. The robot effectively handles complex conditions such as dense fruit tree distribution, terrain undulations, and dynamic obstacles, achieving stable and reliable autonomous inspection operations.

Although the proposed system demonstrates stable navigation performance in standardized orchard environments, its engineering applicability remains constrained by the platform's physical dimensions, sensor update frequency, and battery endurance. Consequently, in scenarios involving high-density planting, rapid interactions with dynamic obstacles, or large-scale long-duration inspections, performance bottlenecks may arise regarding passage clearance, real-time responsiveness, and continuous operational capability. Furthermore, the current study lacks a reproducible quantitative assessment of robustness against typical field disturbances, such as sensor noise and localization drift. To address these challenges, future research will focus on integrating deep learning to enhance perception and dynamic avoidance in complex environments, exploring multi-robot collaboration to improve coverage efficiency and mitigate single-robot endurance constraints, and establishing parameter self-adaptation and online optimization mechanisms to further boost the algorithm's robustness and generalization in unstructured orchards.

Author Contributions: Conceptualization, J.Z. and L.S. (Lepeng Song); methodology, J.Z. and L.S. (Longbo Su); software, L.S. (Longbo Su) and Z.B.; validation, L.S. (Longbo Su); formal analysis, S.X.Y.; investigation, Z.B. and S.H.; funding acquisition, J.Z. and L.S. (Lepeng Song); data curation, Z.B. and S.H.; writing—original draft preparation, J.Z. and L.S. (Longbo Su); writing—review and editing, P.L. and Z.B.; visualization, L.S. (Lepeng Song) and S.X.Y.; supervision, S.X.Y. and W.M.; project administration, W.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research is supported by the Research Foundation of Chongqing University of Science and Technology (Grant No. KJDX2024005).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Teng, H.Z.; Wang, Y.P.; Chatziparaschis, D.; Karydis, K. Adaptive LiDAR Odometry and Mapping for Autonomous Agricultural Mobile Robots in Unmanned Farms. *Comput. Electron. Agric.* **2025**, *232*, 110023. [CrossRef]
2. Zeng, X.L.; Wan, H.; Fan, Z.M.; Yu, X.J.; Guo, H.R. MT-MVSNet: A Lightweight and Highly Accurate Convolutional Neural Network Based on Mobile Transformer for 3D Reconstruction of Orchard Fruit Tree Branches. *Expert Syst. Appl.* **2025**, *268*, 126220. [CrossRef]
3. Barrelmeyer, D.; Stiene, S.; Jose, J.; Porrmann, M. Mobile Ground-Truth 3D Detection Environment for Agricultural Robot Field Testing. *Sensors* **2025**, *25*, 4103. [CrossRef]
4. Luo, Y.; Pu, D.K.; He, X.L.; Song, L.P.; Yang, S.X.; Ma, W.H.; Shi, H.W. Trajectory Tracking Control of an Orchard Robot Based on Improved Integral Sliding Mode Algorithm. *Agriculture* **2025**, *15*, 1881. [CrossRef]
5. Zhu, X.Y.; Gao, Y.F.; Li, Y.Y.; Li, B. Fast Dynamic P-RRT*-Based UAV Path Planning and Trajectory Tracking Control Under Dense Obstacles. *Actuators* **2025**, *14*, 211. [CrossRef]
6. Shen, Y.Y.; Shen, Y.; Zhang, Y.F.; Huo, C.W.; Shen, Z.F.; Su, W.; Liu, H. Research Progress on Path Planning and Tracking Control Methods for Orchard Mobile Robots in Complex Scenarios. *Agriculture* **2025**, *15*, 1917. [CrossRef]
7. Ghambari, S.; Golabi, M.; Jourdan, L.; Lepagnot, J.; Idoumghar, L. UAV Path Planning Techniques: A Survey. *RAIRO Oper. Res.* **2024**, *58*, 2951–2989. [CrossRef]
8. Banik, S.; Banik, S.C.; Mahmud, S.S. Path Planning Approaches in Multi-Robot System: A Review. *Eng. Rep.* **2025**, *7*, e13035. [CrossRef]

9. Guo, H.P.; Li, Y.; Wang, H.; Wang, C.S.; Zhang, J.; Wang, T.W.; Rong, L.R.; Wang, H.Y.; Wang, Z.H.; Huo, Y.B.; et al. Path Planning of Greenhouse Electric Crawler Tractor Based on the Improved A* and DWA Algorithms. *Comput. Electron. Agric.* **2024**, *227*, 109596. [CrossRef]
10. Fu, X.; Huang, Z.C.; Zhang, G.X.; Wang, W.J.; Wang, J. Research on Path Planning of Mobile Robots Based on Improved A* Algorithm. *PeerJ Comput. Sci.* **2025**, *11*, e2691. [CrossRef] [PubMed]
11. Zhang, Y.; Miao, X.Y.; Sun, Y.F.; He, Z.P.; Hou, T.W.; Wang, Z.H.; Wang, Q.Y. Multi-Strategy Fusion RRT-Based Algorithm for Optimizing Path Planning in Continuous Cherry Picking. *Agriculture* **2025**, *15*, 1699. [CrossRef]
12. Huang, J.B.; Chen, C.M.; Shen, J.J.; Liu, G.H.; Xu, F. A Self-Adaptive Neighborhood Search A-Star Algorithm for Mobile Robots Global Path Planning. *Comput. Electr. Eng.* **2025**, *123*, 110018. [CrossRef]
13. Zhu, W.L.; Chen, Z.F. Research on Path Planning for Mobile Charging Robots Based on Improved A* and DWA Algorithms. *Electronics* **2025**, *14*, 2318. [CrossRef]
14. Lei, C.F.; Li, J.; Deng, Y.S.; Tan, X.J. RRTASV: Improved RRT Path Planning Method for Ackermann Steering Vehicles. *Expert Syst. Appl.* **2025**, *279*, 127349. [CrossRef]
15. Dong, X.X.; Wang, Y.J.; Fang, C.; Ran, K.M.; Liu, G.H. FHQ-RRT*: An Improved Path Planning Algorithm for Mobile Robots to Acquire High-Quality Paths Faster. *Sensors* **2025**, *25*, 2189. [CrossRef]
16. Cao, G.Z.; Zhang, B.H.; Li, Y.; Wang, Z.T.; Diao, Z.H.; Zhu, Q.Z.; Liang, Z.Y. Environmental Mapping and Path Planning for Robots in Orchard Based on Traversability Analysis, Improved LeGO-LOAM and RRT* Algorithms. *Comput. Electron. Agric.* **2025**, *230*, 109889. [CrossRef]
17. Sun, H.K.; Zhu, K.; Zhang, W.G.; Ke, Z.F.; Hu, H.H.; Wu, K.; Zhang, T.H. Emergency Path Planning Based on Improved Ant Colony Algorithm. *J. Build. Eng.* **2025**, *100*, 111725. [CrossRef]
18. Wu, D.; Du, K.Y. A Hybrid Optimization Algorithm of ACO and RRT for Solving Mobile Robot Path Planning. *Meas. Sci. Technol.* **2025**, *36*, 076215. [CrossRef]
19. Li, J.; Wan, L.L.; Huang, Z.; Chen, Y.; Tang, H.Y. Hybrid Path Planning Strategy Based on Improved Particle Swarm Optimisation Algorithm Combined with DWA for Unmanned Surface Vehicles. *J. Mar. Sci. Eng.* **2024**, *12*, 1268. [CrossRef]
20. Li, S.Y.; Huang, J.W.; Mao, W.P.; Yang, Y.; Nie, J.W. TOACA-DWA: A Path Planning Algorithm for Mobile Robot in Dynamic Environments. *J. Intell. Fuzzy Syst.* **2025**, *48*, 245–261. [CrossRef]
21. Huang, Z.H.; Ou, C.H.; Guo, Z.P.; Ye, L.; Li, J. Human-Following Strategy for Orchard Mobile Robot Based on the KCF-YOLO Algorithm. *Horticulturae* **2024**, *10*, 348. [CrossRef]
22. Wu, X.; Li, X.W.; Zhang, D.; Li, Z.H.; Zhou, J. A Control Strategy for Point Stabilization and Trajectory Tracking of Wheeled Mobile Robots. *Robot. Auton. Syst.* **2025**, *192*, 105040. [CrossRef]
23. Li, Q.; Tang, H.H.; Ahmad, N.S. Improving Trajectory Tracking of Differential Wheeled Mobile Robots with Enhanced GWO-Optimized Back-Stepping and FOPID Controllers. *IEEE Access* **2025**, *13*, 48872–48887. [CrossRef]
24. Huang, H.J.; Gao, J.F. Backstepping and Novel Sliding Mode Trajectory Tracking Controller for Wheeled Mobile Robots. *Mathematics* **2024**, *12*, 1458. [CrossRef]
25. Yiğit, S.; Sezgin, A. Trajectory Planning with A* Algorithm and Tracking with Multiple Controllers. *Optim. Control Appl. Methods* **2025**, *46*, 1036–1048. [CrossRef]
26. Olmez, Y.; Koca, G.O.; Akpolat, Z.H. Clonal Selection Algorithm Based Control for Two-Wheeled Self-Balancing Mobile Robot. *Simul. Model. Pract. Theory* **2022**, *118*, 102552. [CrossRef]
27. Fan, X.L.; Bu, C.G.; Zhao, X.G.; Sui, J.; Mo, H.W. Incremental Double Q-Learning-Enhanced MPC for Trajectory Tracking of Mobile Robots. *IEEE Trans. Instrum. Meas.* **2025**, *74*, 2509414. [CrossRef]
28. Liu, Y.F.; Hu, T.; Guan, X.Q.; Wang, Y.X.; Zhang, B.X.; Wang, Y.; Li, G. Adaptive MPC-Based Multi-Terrain Trajectory Tracking Framework for Mobile Spherical Robots. *IEEE/ASME Trans. Mechatron.* **2025**, *30*, 6785–6797. [CrossRef]
29. Tang, J.W.; Wu, S.; Lan, B.; Dong, Y.H.; Jin, Y.Q.; Tian, G.J.; Zhang, W.A.; Shi, L. GMPC: Geometric Model Predictive Control for Wheeled Mobile Robot Trajectory Tracking. *IEEE Robot. Autom. Lett.* **2024**, *9*, 4822–4829. [CrossRef]
30. Zhang, T.; Jiao, X.H.; Li, X.K. Improved Adaptive Iterative Learning Trajectory Tracking Control with Adaptive High-Order Internal Model for Autonomous Farming Vehicle. *Comput. Electron. Agric.* **2025**, *236*, 110444. [CrossRef]
31. Xu, C.B.; Zhou, X.Y.; Chen, R.P.; Li, B.Z.; He, W.H.; Li, Y.; Ye, F.P. Trajectory Tracking for 3-Wheeled Independent Drive and Steering Mobile Robot Based on Dynamic Model Predictive Control. *Appl. Sci.* **2025**, *15*, 485. [CrossRef]
32. Ma, W.H.; Wang, X.M.; Yang, S.X.; Xue, X.L.; Li, M.Y.; Wang, R.; Yu, L.G.; Song, L.P.; Li, Q.F. Autonomous Inspection Robot for Dead Laying Hens in Caged Layer House. *Comput. Electron. Agric.* **2024**, *227*, 109595. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.